

École Nationale des Sciences Appliquées de Tétouan

Filière: Big Data & Intelligence Artificielle

Génération de légendes d'images avec des modèles profonds

Une approche basée sur l'apprentissage automatique

Réalisé par :

- Abdessamad Ait Brahim
- Ayyoub Amazal
- Yassir Lagnaoui

Encadré par :

- Dr. Imane Hachchane

Année Académique : 2024-2025

Remerciement

Nous tenons à exprimer notre profonde gratitude à notre professeur et encadrant, Madame **Imane Hachchane**, pour sa disponibilité, sa patience et la richesse de ses conseils tout au long de ce projet.

Son expertise, sa rigueur scientifique et son implication pédagogique ont grandement contribué à l'aboutissement de ce travail. Son accompagnement attentif et ses orientations précieuses nous ont permis de surmonter les obstacles rencontrés et de progresser aussi bien sur le plan technique que personnel.

Ce projet a été pour nous une véritable opportunité d'apprentissage, d'exploration intellectuelle et de collaboration, que nous avons menée avec enthousiasme et détermination, guidés par ses encouragements et sa vision claire.

Nous tenons également à souligner l'importance de son mentorat qui a dépassé le simple cadre académique, nous inculquant des valeurs de persévérance, d'excellence et d'intégrité scientifique qui nous accompagneront bien au-delà de cette formation.

« Comme tous ceux qui résistent patiemment face à notre ignorance,
vous méritez plus que de la reconnaissance... vous méritez notre respect. »

Avec notre sincère reconnaissance,
Les étudiants du groupe

Table des matières

1 Résumé **3**

2 Abstract **3**

3 Introduction **4**

4 Méthodologie **4**
4.1 Dataset . . . 4
4.2 Source du Dataset . . . 5
4.3 Outils utilisés . . . 6
4.4 Exploration des données . . . 8

5 Modèles utilisés et architectures **9**
5.1 Architecture VGG19 + LSTM pour la génération de légendes d'images . . . 9
5.1.1 Vue d'ensemble . . . 9
5.1.2 Encodeur : VGG19 avec max Pooling . . . 9
5.1.3 Décodeur : LSTM avec Embedding . . . 11
5.1.4 Prétraitement et apprentissage . . . 12
5.1.5 Hyperparamètres . . . 12
5.1.6 Processus de génération . . . 12
5.2 CNN InceptionV3 + BiLSTM + Mécanisme d'attention . . . 12
5.2.1 Description détaillée de l'architecture . . . 13
5.2.2 Encodeur InceptionV3 . . . 13
5.2.3 Décodeur BiLSTM . . . 14
5.2.4 Mécanisme d'attention . . . 14
5.2.5 Processus d'apprentissage . . . 15
5.2.6 Avantages de cette architecture . . . 16
5.3 CNN InceptionV3 + Transformers . . . 16
5.3.1 Motivation . . . 17
5.3.2 Architecture détaillée . . . 17
5.3.3 Entraînement . . . 19
5.3.4 Inférence . . . 19
5.3.5 Avantages de cette architecture . . . 20
5.3.6 Mécanisme d'attention croisée image-texte . . . 21
5.3.7 Fonctionnement simplifié . . . 21
5.3.8 Comportement observé . . . 21
5.3.9 Avantages du mécanisme d'attention . . . 22
5.4 Modèle préentraîné BLIP . . . 22
5.4.1 Principes fondamentaux . . . 22
5.4.2 Architecture détaillée . . . 23
5.4.3 Processus de pré-entraînement . . . 23
5.4.4 Performances et résultats . . . 24
5.4.5 Adaptabilité et fine-tuning . . . 24
5.4.6 Avantages techniques . . . 25
5.4.7 Implémentation pratique . . . 25
5.4.8 Applications concrètes . . . 25

5.4.9 Limites et défis 26
5.4.10 Évolutions et perspectives futures 26

6 Entraînement et validation 27

7 Résultats et analyse 27
7.1 Évaluation des performances 27
7.1.1 Le score BLEU (Bilingual Evaluation Understudy) 27
7.1.2 Résultats comparatifs 28

8 Déploiement 29

9 Simulation 29
9.1 Interface de simulation 30
9.2 Comparaison des modèles 30
9.3 Modèle VGG19 + LSTM 31
9.4 Modèle InceptionV3 + BLSTM 32
9.5 Modèle BLIP 33
9.6 Analyse des résultats 34

10 Conclusion 34
10.1 Conclusion générale 34
10.2 Limites identifiées 36

11 Perspectives 36
11.1 Perspectives générales 36
11.2 Perspectives spécifiques au dataset Flickr 32K 37

12 Annexes : Les références 39

1 Résumé

Ce projet explore la tâche de *image captioning*, un domaine fascinant du machine learning qui consiste à générer automatiquement des descriptions textuelles pour des images données. Cette tâche se situe à l'intersection de deux disciplines clés de l'intelligence artificielle . Elle présente des défis importants, notamment la capacité à comprendre le contenu visuel complexe d'une image et à le traduire en une phrase grammaticalement correcte, contextuellement pertinente et sémantiquement fidèle. Nous présentons une approche novatrice qui combine la puissance des réseaux neuronaux convolutifs (CNN), tels que VGG16, InceptionV3, et transformateurs, pour l'extraction de caractéristiques visuelles riches, avec la flexibilité des réseaux neuronaux récurrents (RNN), en particulier les Long Short-Term Memory (LSTM), ainsi que l'architecture Transformer, pour la génération de séquences de mots cohérentes. Ce cadre permet au modèle de capturer à la fois les relations spatiales dans l'image et les dépendances linguistiques dans le texte. L'objectif principal est de développer un système intelligent capable de comprendre le contenu sémantique d'une image et de générer une légende descriptive en langage naturel, reflétant les objets présents, leurs relations et le contexte global. Des techniques d'optimisation telles que l'attention (attention mechanism) sont également intégrées afin de focaliser le modèle sur les régions visuellement importantes lors de la génération du texte. Les résultats attendus comprennent un modèle performant, capable de produire des légendes précises, grammaticalement correctes et riches sur le plan contextuel. Ce travail ouvre la voie à des applications concrètes telles que l'accessibilité pour les personnes malvoyantes, la recherche d'images, l'annotation automatique de contenu multimédia, ou encore la robotique intelligente.

2 Abstract

This project explores the task of image captioning, a fascinating area of machine learning that focuses on automatically generating textual descriptions for given images. This task lies at the intersection of two key disciplines in artificial intelligence : computer vision and natural language processing. It presents significant challenges, particularly the ability to understand the complex visual content of an image and translate it into a grammatically correct, contextually relevant, and semantically accurate sentence. We propose an innovative approach that combines the power of Convolutional Neural Networks (CNNs), such as VGG16, InceptionV3, and Transformer-based models, for extracting rich visual features, with the flexibility of Recurrent Neural Networks (RNNs)—particularly Long Short-Term Memory (LSTM) networks—as well as the Transformer architecture for generating coherent word sequences. This framework enables the model to capture both the spatial relationships within the image and the linguistic dependencies in the generated text. The main objective is to develop an intelligent system capable of understanding the semantic content of an image and producing a descriptive caption in natural language that reflects the objects present, their relationships, and the overall context. Optimization techniques such as the attention mechanism are also integrated to allow the model to focus on visually salient regions when generating the caption. The project aims to deliver a high-performing model that generates accurate and contextually relevant captions, enabling real-world applications such as accessibility for the visually impaired, image retrieval, content annotation, and intelligent robotics.

3 Introduction

L'intelligence artificielle (IA) connaît une évolution rapide, repoussant constamment les frontières des capacités humaines et des machines. Parmi les avancées les plus impressionnantes figure la convergence entre la vision par ordinateur et le traitement du langage naturel, ouvrant la voie à des applications révolutionnaires. L'une des plus captivantes est l'image captioning, qui permet à une machine de générer des descriptions textuelles d'images de manière autonome.

Cette technologie repose sur l'idée que, pour comprendre une image, il ne suffit pas de la visualiser, mais qu'il faut également pouvoir la décrire de manière cohérente et naturelle. Imaginez un système capable de regarder une scène complexe, d'en identifier les éléments clés, d'en saisir les relations, et de retranscrire cette compréhension dans une phrase en langage naturel. Ce processus engage une combinaison de techniques avancées, notamment l'utilisation de réseaux neuronaux convolutifs (CNN) pour l'extraction des caractéristiques visuelles des images, et de réseaux neuronaux récurrents (RNN), tels que les Long Short-Term Memory (LSTM) et l'architecture Transformer, pour la génération du langage. L'objectif de ce projet est de construire un modèle de captioning qui puisse, à partir d'une image, produire une légende textuelle non seulement précise et grammaticalement correcte, mais également riche en contexte et en sens. Cela nécessite une approche hybride combinant à la fois des méthodes de vision par ordinateur et de traitement du langage naturel. En utilisant les CNN pour analyser le contenu visuel de l'image et les RNN pour générer un texte fluide, ce modèle devra être capable de comprendre les objets présents, leurs interactions et le contexte global de la scène, tout en respectant les subtilités du langage humain. Les implications d'un tel système sont vastes. Il pourrait révolutionner l'accessibilité, en offrant aux malvoyants la possibilité de "voir" grâce à des descriptions générées par l'IA. Dans la recherche d'images, il permettrait d'explorer et d'organiser des collections via des requêtes en langage naturel. Plus largement, l'image captioning pourrait favoriser la création de machines plus intelligentes, capables d'interagir avec leur environnement de manière fluide, en combinant informations visuelles et linguistiques. Ce projet représente à la fois un défi théorique et une application concrète de l'apprentissage automatique, à travers l'analyse d'architectures neuronales, l'expérimentation de modèles existants et l'évaluation rigoureuse de leurs performances, afin de faire avancer le domaine du captioning d'images et d'enrichir les perspectives de recherche.

4 Méthodologie

4.1 Dataset

Le dataset **Flickr30k** a été sélectionné en raison de sa richesse sémantique, de sa structure bien annotée et de sa large adoption dans la recherche sur la génération de légendes d'images. Il contient environ 31 000 images issues de la plateforme Flickr, chacune accompagnée de cinq descriptions textuelles rédigées manuellement par des annotateurs humains. Cette pluralité d'annotations permet de capturer différentes interprétations possibles d'une même scène, ce qui est essentiel pour entraîner des modèles capables de générer des descriptions variées et contextuellement appropriées.

Motivations du choix Flickr30k est considéré comme un benchmark de référence dans la communauté scientifique en vision par ordinateur et en traitement du langage naturel. Son adoption dans de nombreux travaux antérieurs permet une évaluation rigoureuse, reproductible et directement comparable avec les approches de l'état de l'art. Contrairement à d'autres datasets plus vastes comme MS COCO, Flickr30k offre un compromis optimal entre la taille du corpus et la qualité des annotations.

Le dataset couvre une large variété de scènes visuelles, notamment des activités humaines, des interactions sociales, des animaux, des objets du quotidien et des paysages urbains ou naturels. Cette diversité favorise l'apprentissage de représentations visuelles généralisables et réduit les biais contextuels. De plus, l'existence de cinq descriptions différentes pour chaque image permet d'entraîner des modèles robustes aux variations lexicales et syntaxiques, augmentant ainsi la capacité du générateur à produire des légendes plus riches et naturelles.

4.2 Source du Dataset

Le dataset est disponible publiquement sur la plateforme Kaggle à l'adresse suivante : https://www.kaggle.com/datasets/adityajn105/flickr30k

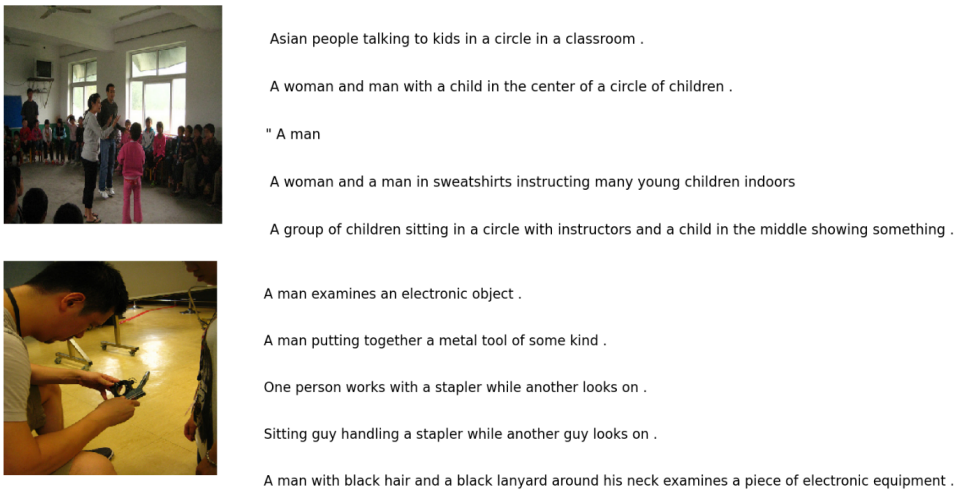


FIGURE 1 – Exemple d'images extraites du dataset Flickr30k avec leurs descriptions associées

Atouts du dataset

- **Annotations humaines de haute qualité :** chaque légende est écrite par un humain, garantissant une richesse linguistique et une pertinence contextuelle.
- **Multiplicité des légendes par image :** cinq descriptions par image offrent différentes perspectives d'interprétation, ce qui est bénéfique pour la génération de texte.
- **Taille modérée et bien équilibrée :** permet un entraînement efficace même avec des ressources limitées, tout en assurant une couverture suffisante pour capter la complexité des scènes.

— **Dataset multilingue potentiellement extensible :** bien que les légendes soient en anglais, la structure du dataset le rend compatible avec des expériences de traduction automatique ou de génération multilingue.

Prétraitement appliqué Avant l'entraînement, un ensemble de traitements a été appliqué aux images et aux légendes :

— **Nettoyage des légendes** : suppression de la ponctuation, conversion en minuscules, normalisation des caractères spéciaux.

— **Ajout de tokens spéciaux** : insertion de tokens `<start>` et `<end>` autour de chaque légende pour encadrer la séquence générée.

— **Tokenisation** : utilisation d'un *Tokenizer Keras* pour transformer les légendes textuelles en séquences numériques.

— **Filtrage du vocabulaire** : limitation du vocabulaire aux 10 000 mots les plus fréquents pour équilibrer richesse lexicale et efficacité computationnelle.

— **Traitement des images** : redimensionnement des images en fonction du modèle CNN utilisé :

  — 224×224 pixels pour VGG19

  — 299×299 pixels pour InceptionV3

  suivi d'une normalisation adaptée au modèle pré-entraîné (soustraction de la moyenne ou mise à l'échelle entre [0,1]).

Répartition des données Le dataset a été divisé selon une stratégie classique favorisant la robustesse de l'évaluation :

— **Ensemble d'entraînement :** 28 000 (90%) pour l'apprentissage du modèle.

— **Ensemble de test :** 3 000 images (10%) réservées à l'évaluation finale et à la comparaison avec les travaux existants.

4.3 Outils utilisés

La réalisation de ce projet repose sur une suite d'outils logiciels et bibliothèques spécialisées dans le traitement d'images, le traitement du langage naturel (NLP), l'apprentissage profond, ainsi que le prototypage rapide et le déploiement web. Le choix de ces outils vise à assurer à la fois la performance, la modularité, la reproductibilité, et la facilité d'intégration des différents composants du pipeline.

— **Environnement de développement** :

  — **Python** , langage principal pour l'ensemble du projet en raison de sa richesse en bibliothèques scientifiques et sa forte communauté.

  — **Kaggle env**, utilisé comme plateforme de développement collaborative et hébergeant les notebooks d'entraînement, avec l'accès à un GPU *Tesla V100* permettant l'accélération du traitement et de l'entraînement des modèles de deep learning.

  — **Google Drive**, intégré à Colab pour la gestion des datasets lourds et le stockage des checkpoints de modèles.

- **Frameworks d'apprentissage profond** :
  - **TensorFlow** et **Keras** : pour la conception, la compilation et l'entraînement du modèle image-captioning basé sur CNN+LSTM. L'API Keras simplifie la définition d'architectures complexes tout en offrant un backend puissant.
  - **PyTorch** : utilisé spécifiquement pour expérimenter avec des modèles préentraînés récents, notamment **BLIP (Bootstrapped Language Image Pretraining)**, facilitant l'intégration d'architectures transformer pour la génération d'images légendées.
- **Architectures de modèles utilisées** :
  - **CNNs préentraînés** : VGG19 et InceptionV3 — chargés depuis `keras.applications`, utilisés comme extracteurs de caractéristiques visuelles. Chaque modèle impose des contraintes spécifiques de taille d'entrée et de normalisation (224×224 pour VGG16, 299×299 pour InceptionV3).
  - **Réseaux LSTM** : pour la génération des séquences textuelles, intégrés à un module de décodage séquentiel. L'architecture comprend une couche d'embedding, une ou plusieurs couches LSTM, et une couche dense finale avec activation softmax.
  - **Modèles à attention (optionnels)** : mécanisme d'attention ajouté pour pondérer dynamiquement les caractéristiques visuelles à chaque pas de génération.
- **Traitement du langage naturel (NLP)** :
  - **NLTK** : utilisé pour le prétraitement linguistique (tokenisation, normalisation, suppression des mots vides, etc.).
  - **Keras Tokenizer** : pour transformer les séquences textuelles en vecteurs numériques. Ce tokenizer construit un dictionnaire $mot \rightarrow index$ et encode les phrases sous forme de séquences indexées, utilisées comme entrées pour le modèle LSTM.
  - **Transformers (Hugging Face)** : utilisés pour comparer avec des modèles préentraînés comme BERT ou T5, ou pour intégrer un encodeur texte plus avancé dans les variantes du pipeline (BLIP, CLIP, etc.).
- **Manipulation et visualisation des données** :
  - **NumPy** et **Pandas** : pour la gestion des tableaux de données, le traitement des matrices d'images, et le nettoyage des métadonnées textuelles.
  - **Matplotlib**, **Seaborn** : pour l'analyse exploratoire, la visualisation des distributions, et le suivi de l'apprentissage (pertes, métriques, attention maps).
  - **TensorBoard** : pour le suivi visuel de l'entraînement (perte, précision, métriques personnalisées comme BLEU ou CIDEr).
- **Déploiement web et prototypage** :
  - **Flask** : framework minimaliste pour créer une interface web permettant d'interagir avec le modèle. L'utilisateur peut y téléverser une image et obtenir une légende générée dynamiquement.
  - **HTML/CSS/JS** : pour enrichir l'interface et faciliter la démonstration interactive du système.

4.4 Exploration des données

Avant toute phase d'entraînement, une analyse exploratoire approfondie a été réalisée afin de mieux comprendre les caractéristiques statistiques, linguistiques et visuelles du dataset *Flickr30k*. Cette étape est cruciale pour orienter les choix d'architecture, de nettoyage et de dimensionnement des paramètres du modèle.

- **Analyse des légendes** :
  - **Longueur des descriptions** : histogramme des longueurs de séquences montre que la plupart des légendes contiennent entre 6 et 14 mots. Cela a guidé la décision de fixer la longueur maximale des séquences à 34 tokens (avec padding).
  - **Vocabulaire** : environ 20 000 mots uniques. Après filtrage des mots rares, un vocabulaire réduit à 10 000 mots les plus fréquents a été conservé.
  - **Analyse des fréquences** : les mots les plus fréquents sont « man », « woman », « child », « dog », « playing », « standing », « wearing », ce qui révèle la nature descriptive du dataset centrée sur des scènes humaines et animalières.
  - **Nettoyage linguistique** : suppression de la ponctuation, mise en minuscules, et ajout des tokens spéciaux `<start / startseq>` et `<end / endseq>`.
- **Utilisation du tokenizer Keras** :
  - Création d'un mapping indexé `word → id` et `id → word`.
  - Encodage des phrases sous forme de vecteurs de longueur fixe, avec *post-padding*.
  - Sauvegarde du tokenizer pour l'inférence future.
- **Analyse des images** :
  - Les dimensions originales des images varient considérablement : un redimensionnement est nécessaire pour uniformiser les entrées des CNN.
  - Analyse de la distribution des valeurs RGB : histogrammes montrant des images bien exposées, sans biais de couleur ou surreprésentation de teintes.
  - Exemples d'images : des scènes contenant des activités humaines, du sport, des animaux, des véhicules, etc.
- **Prétraitement des images** :
  - **Redimensionnement** :
    - **224×224** pour VGG19
    - **299×299** pour InceptionV3
  - **Normalisation** :
    - Mise à l'échelle des pixels dans [0,1] pour certains modèles.
    - Centrage selon la moyenne ImageNet pour d'autres (ex : VGG19).
  - **Augmentation de données (Data Augmentation)** :
    - `ImageDataGenerator` de Keras appliqué à l'ensemble d'entraînement : rotations, zooms, translations, inversions horizontales, recadrages.
    - Objectif : améliorer la généralisation du modèle en exposant celui-ci à des variations d'apparence réalistes.
- **Prétraitement des légendes** :
  - Transformation systématique : suppression des caractères spéciaux, conversion en minuscules, filtrage des tokens rares.
  - Ajout de `<start>` et `<end>` pour guider le processus de génération.
  - Construction d'un dictionnaire de vocabulaire, sauvegardé pour l'évaluation et le déploiement.

5 Modèles utilisés et architectures

5.1 Architecture VGG19 + LSTM pour la génération de légendes d'images

Le modèle suit l'architecture classique *encoder-decoder*, largement utilisée dans les tâches de traitement de séquences conditionnées sur des entrées visuelles, comme la traduction d'image en texte. Cette approche permet de convertir une image en une séquence de mots formant une légende pertinente.

5.1.1 Vue d'ensemble

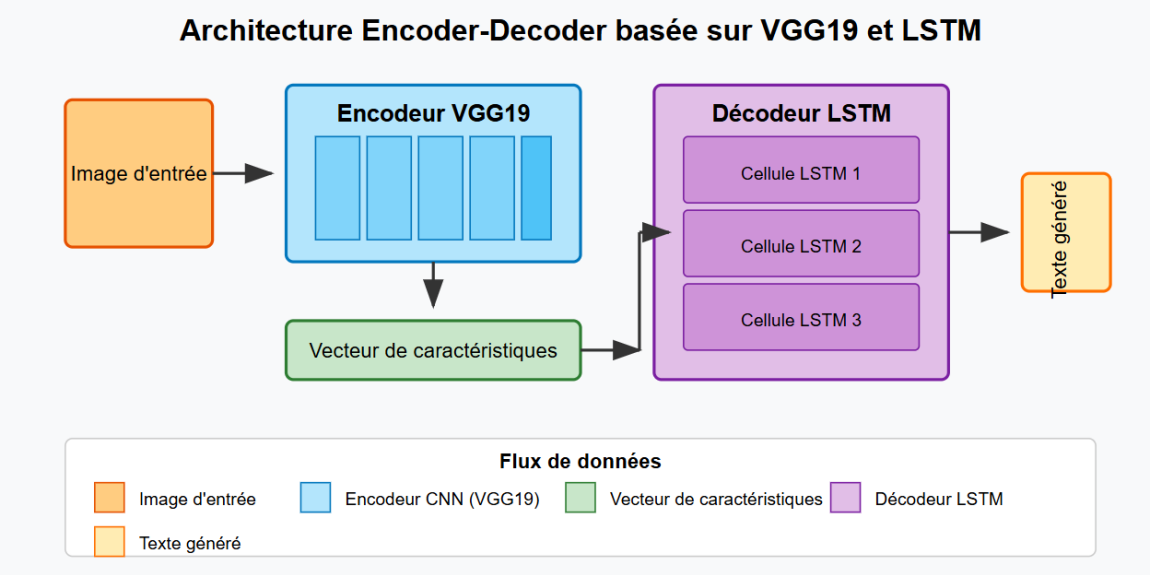


FIGURE 2 – Architecture Encoder-Decoder (VGG19 + LSTM)

L'architecture globale est constituée de deux composants majeurs :

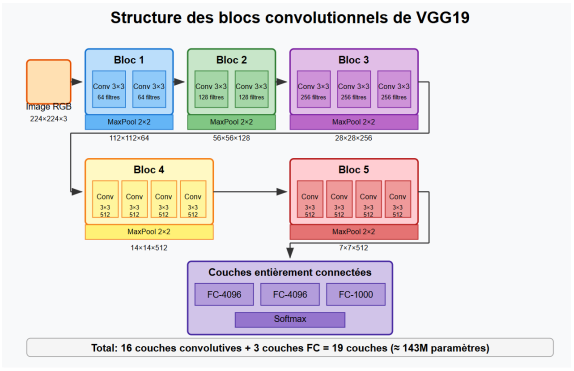
— **Encodeur (VGG19)** : un réseau convolutionnel pré-entraîné servant à extraire les caractéristiques visuelles importantes d'une image. Il génère un vecteur $\mathbf{v} \in R^{512}$.

— **Décodeur (LSTM)** : un modèle séquentiel qui utilise ce vecteur $\mathbf{v}$ pour générer mot par mot une description textuelle cohérente de l'image.

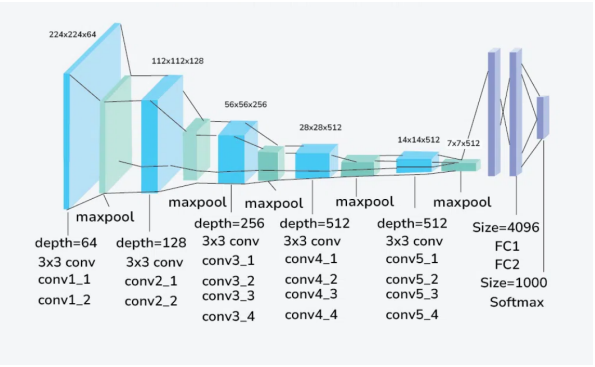
Ce type de modèle apprend à "traduire" des images vers des descriptions textuelles, une tâche complexe nécessitant une compréhension conjointe de la vision et du langage.

5.1.2 Encodeur : VGG19 avec max Pooling

Le modèle VGG19, connu pour sa profondeur et sa performance en classification d'images, est utilisé ici comme extracteur de caractéristiques. Les couches pleinement connectées sont supprimées pour conserver uniquement les couches convolutives. Cela permet de garder une représentation spatiale et hiérarchique de l'image.



(a) Structure des blocs convolutionnels de VGG19



(b) Architecture de VGG19

FIGURE 3 – Présentation générale de l'architecture VGG19

Détails des blocs VGG19

- **Bloc 1 :** Ce bloc contient deux couches de convolution avec 64 filtres chacune, de taille 3×3, accompagnées d'une fonction d'activation ReLU après chaque convolution. Ensuite, une opération de MaxPooling 2×2 est appliquée afin de réduire la taille spatiale de l'image.
- **Bloc 2 :** Deux couches de convolution avec 128 filtres 3×3, suivies chacune de ReLU. Un MaxPooling est appliqué pour réduire la taille des cartes de caractéristiques.
- **Bloc 3 :** Ce bloc comprend quatre couches convolutionnelles avec 256 filtres 3×3, chacune suivie d'une activation ReLU. Un MaxPooling est ensuite utilisé pour compresser les dimensions spatiales.
- **Bloc 4 :** Quatre couches de convolution avec 512 filtres 3×3 sont utilisées, avec ReLU après chaque convolution. Une opération de MaxPooling est à nouveau appliquée.
- **Bloc 5 :** Identique au bloc précédent, il contient quatre couches de convolution avec 512 filtres 3×3 et ReLU. Il est également suivi d'un MaxPooling.

Max Pooling Le *Max Pooling* est une opération de sous-échantillonnage qui consiste à diviser chaque carte de caractéristiques en petites régions (généralement 2×2) et à ne conserver que la valeur maximale de chaque région. Cela permet :

- de réduire la dimensionnalité spatiale des cartes de caractéristiques ;
- de conserver les activations les plus significatives ;
- d'introduire une forme d'invariance locale aux translations.

Vecteur de caractéristiques Le vecteur de sortie est calculé comme suit :

$$\mathbf{v} = \text{Flatten}(\text{MaxPool}(\text{VGG19}_{\text{features}}(I))) \in R^{25088}$$

où I représente l'image d'entrée, et $\text{VGG19}_{\text{features}}(I) \in R^{512 \times 7 \times 7}$ est la sortie du dernier bloc convolutionnel. Le *Flatten* transforme ce tenseur 3D en un vecteur 1D. Le vecteur $\mathbf{v}$ capture les caractéristiques visuelles extraites par les filtres convolutifs, prêtes à être exploitées pour la génération de légendes.

5.1.3 Décodeur : LSTM avec Embedding

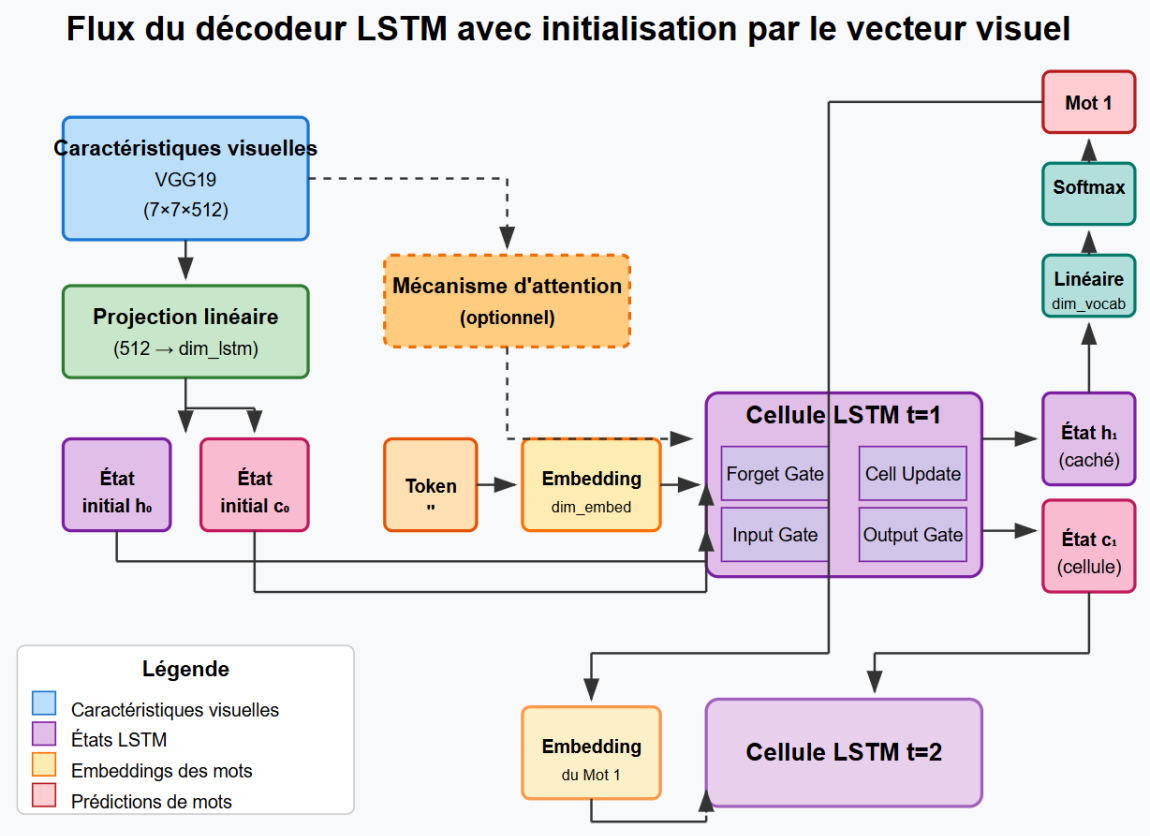


FIGURE 4 – Flux du décodeur LSTM avec initialisation par le vecteur visuel

Embedding des mots Chaque mot du vocabulaire est converti en un vecteur dense (embedding), ce qui permet de capturer la sémantique du mot :

$$\mathbf{e}_t = \text{Embed}(w_t), \quad \mathbf{e}_t \in R^{256}$$

Initialisation du LSTM Le vecteur visuel $\mathbf{v}$ est utilisé pour initialiser l'état caché $\mathbf{h}_0$ et l'état de mémoire $\mathbf{c}_0$ du LSTM :

$$\mathbf{h}_0 = \tanh(W_h\mathbf{v}), \quad \mathbf{c}_0 = \tanh(W_c\mathbf{v})$$

Ces états initiaux intègrent la connaissance visuelle dans le modèle séquentiel.

Génération séquentielle À chaque étape t, on concatène l'embedding $\mathbf{e}_t$ avec le vecteur visuel $\mathbf{v}$:

$$\mathbf{x}_t = [\mathbf{e}_t; \mathbf{v}]$$

Le LSTM traite cette entrée et produit un vecteur caché $\mathbf{h}_t$:

$$\mathbf{h}_t, \mathbf{c}_t = \text{LSTM}(\mathbf{x}_t, (\mathbf{h}_{t-1}, \mathbf{c}_{t-1}))$$

Puis on applique une couche linéaire suivie d'une softmax pour obtenir la distribution de probabilité sur les mots :

$$\hat{y}_t = \text{Softmax}(W_o\mathbf{h}_t + b_o)$$

Le mot avec la plus haute probabilité est sélectionné comme w_t.

5.1.4 Prétraitement et apprentissage

Images : elles sont redimensionnées à 224×224 pixels pour s'adapter à l'entrée de VGG19, puis normalisées avec les moyennes et écarts-types d'ImageNet.

Légendes : chaque phrase est tokenisée (découpée en mots), convertie en minuscules, nettoyée des symboles inutiles, puis encodée avec les tokens spéciaux : `<start>`, `<end>`, `<unk>`, `<pad>`.

Fonction de perte : On utilise l'entropie croisée catégorique entre les mots réels w_t et les mots prédits $\hat{y}_t$:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(w_t \mid w_{1:t-1}, \mathbf{v})$$

Optimisation : la descente de gradient est effectuée avec l'algorithme **Adam**, avec un taux d'apprentissage $lr = 0.001$. Le modèle est entraîné avec le mode *teacher forcing* : à chaque étape, le mot réel est donné en entrée au LSTM.

5.1.5 Hyperparamètres

Paramètre	Valeur	Remarque
Embedding dimension	256	Représentation vectorielle des mots
Taille LSTM	512	Capacité à mémoriser le contexte
Batch size	64–128	Entraînement efficace
Dropout	0.3–0.5	Réduction du surapprentissage

5.1.6 Processus de génération

1. L'image est prétraitée (redimensionnée et normalisée).
2. Les caractéristiques $\mathbf{v}$ sont extraites via VGG19.
3. $\mathbf{h}_0$ et $\mathbf{c}_0$ sont initialisés à partir de $\mathbf{v}$.
4. Le token `<start>` est donné comme premier mot.
5. À chaque étape t, le mot w_t est prédit à partir du mot w_{t-1}.
6. La génération s'arrête lorsque le token `<end>` est généré ou qu'une longueur maximale est atteinte.

5.2 CNN InceptionV3 + BiLSTM + Mécanisme d'attention

La deuxième architecture combine la puissance du réseau préentraîné InceptionV3 pour l'extraction de caractéristiques visuelles avec un réseau LSTM bidirectionnel (BiLSTM) pour la génération de descriptions textuelles. Cette approche améliore significativement la capacité du modèle à capturer les dépendances temporelles dans les deux directions , par suite l'utilisation le mécanisme d'attention permet de focaliser dynamiquement sur différentes régions de l'image pendant la génération de texte .

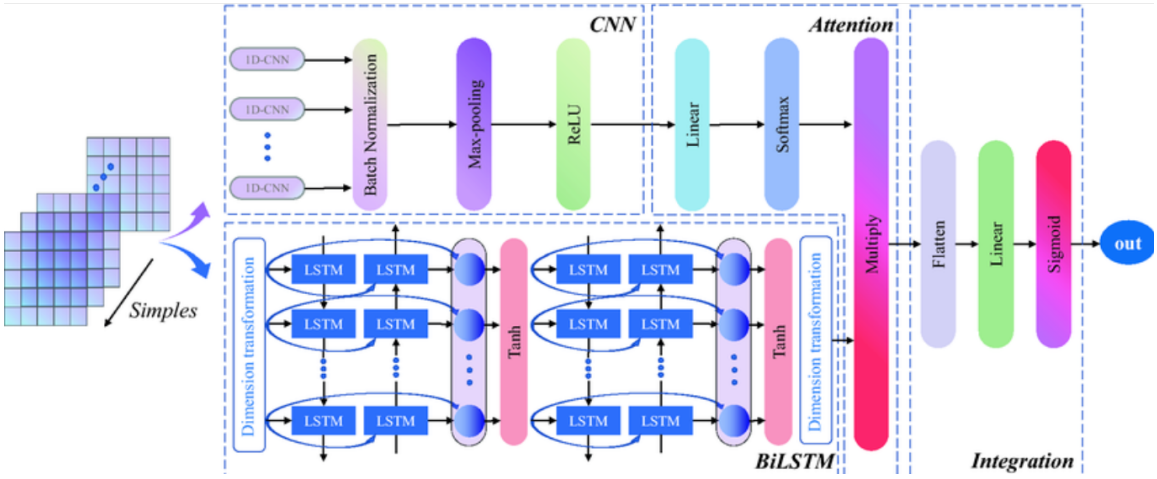


FIGURE 5 – Architecture de CNN InceptionV3 + BiLSTM + Mécanisme d'attention

5.2.1 Description détaillée de l'architecture

Architecture globale :

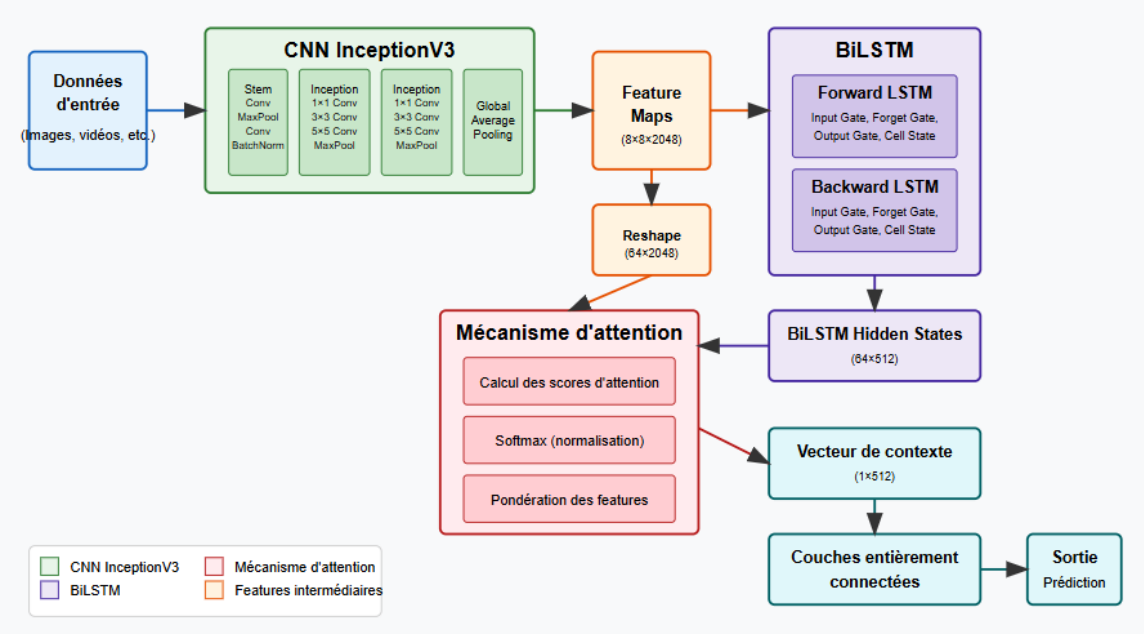


FIGURE 6 – Architecture Globale

— **Encodeur** : InceptionV3 (préentraîné sur ImageNet) sans la couche de classification, suivi d'une projection linéaire pour adapter la dimension des caractéristiques
— **Décodeur** : Couche d'embedding de mots (dimension 300), BiLSTM (512 unités dans chaque direction), couche Dense avec activation softmax (taille du vocabulaire)
— **Mécanisme d'attention** : Attention multiplicative entre les caractéristiques spatiales de l'image et les états cachés du BiLSTM

5.2.2 Encodeur InceptionV3

L'architecture InceptionV3 représente une évolution significative par rapport à VGG19 en termes d'efficacité et de performance. Elle utilise des modules Inception qui combinent

des convolutions à différentes échelles (1×1, 3×3, 5×5) afin de capturer des informations à différents niveaux d'abstraction simultanément.

Contrairement à VGG19, InceptionV3 :

— Réduit considérablement le nombre de paramètres (environ 24M contre 143M pour VGG19)

— Utilise des convolutions factorisées qui décomposent les filtres nxn en combinaisons de 1×n et n×1

— Intègre des convolutions auxiliaires qui aident à combattre le problème de disparition du gradient

— Emploie la normalisation par lots (batch normalization) pour stabiliser l'apprentissage

Dans notre modèle, nous extrayons les caractéristiques du dernier bloc convolutional d'InceptionV3 avant la couche de classification, ce qui produit un tenseur de forme $8 \times 8 \times 2048$. Nous appliquons ensuite une projection linéaire pour obtenir un ensemble de L vecteurs de caractéristiques de dimension D :

$$\mathbf{V} = \{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_L\}, \mathbf{v}_i \in R^D \tag{1}$$

où $L = 64$ (8×8) et $D = 512$.

5.2.3 Décodeur BiLSTM

Le décodeur utilise un LSTM bidirectionnel qui traite l'information dans les deux sens (avant et arrière), permettant ainsi de capturer des dépendances à court et à long terme dans les deux directions. Pour chaque mot w_t de la séquence, le BiLSTM calcule :

$$\overrightarrow{\mathbf{h}}_t = \overrightarrow{\text{LSTM}}(\mathbf{x}_t, \overrightarrow{\mathbf{h}}_{t-1}, \overrightarrow{\mathbf{c}}_{t-1}) \tag{2}$$

$$\overleftarrow{\mathbf{h}}_t = \overleftarrow{\text{LSTM}}(\mathbf{x}_t, \overleftarrow{\mathbf{h}}_{t+1}, \overleftarrow{\mathbf{c}}_{t+1}) \tag{3}$$

$$\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \tag{4}$$

où $\mathbf{x}_t$ est l'embedding du mot d'entrée à l'instant t, $\overrightarrow{\mathbf{h}}_t$ et $\overleftarrow{\mathbf{h}}_t$ sont les états cachés des LSTM avant et arrière, et $\overrightarrow{\mathbf{c}}_t$ et $\overleftarrow{\mathbf{c}}_t$ sont les états de cellule correspondants.

L'état caché final $\mathbf{h}_t$ est la concaténation des états cachés avant et arrière, ce qui donne un vecteur de dimension 1024 (2×512).

5.2.4 Mécanisme d'attention

Le mécanisme d'attention permet au décodeur de se concentrer sur différentes parties de l'image à chaque pas de temps lors de la génération de la description. L'attention multiplicative calcule un score de compatibilité entre chaque vecteur de caractéristiques visuelles $\mathbf{v}_i$ et l'état caché actuel du BiLSTM $\mathbf{h}_{t-1}$:

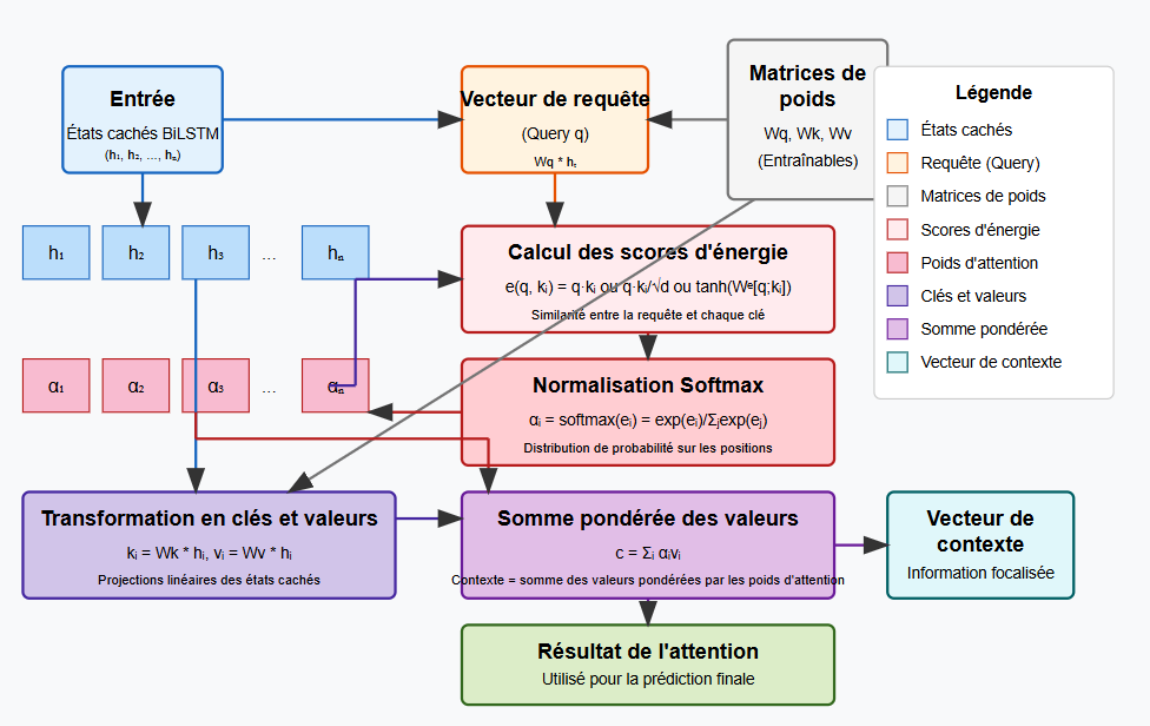


FIGURE 7 – Architecture de Mécanisme d'attention

$$e_{ti} = \mathbf{h}_{t-1}^T \mathbf{W}_a \mathbf{v}_i \tag{5}$$

où $\mathbf{W}_a \in R^{1024\times512}$ est une matrice de poids apprise.

Ces scores sont ensuite normalisés à l'aide d'une fonction softmax pour obtenir les poids d'attention :

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^{L} \exp(e_{tj})} \tag{6}$$

Le vecteur de contexte est calculé comme une somme pondérée des vecteurs de caractéristiques :

$$\mathbf{c}_t = \sum_{i=1}^{L} \alpha_{ti} \mathbf{v}_i \tag{7}$$

Ce vecteur de contexte est ensuite concaténé à l'état caché du BiLSTM pour prédire le mot suivant :

$$\mathbf{y}_t = \text{softmax}(\mathbf{W}_o[\mathbf{h}_t; \mathbf{c}_t] + \mathbf{b}_o) \tag{8}$$

où $\mathbf{W}_o$ et $\mathbf{b}_o$ sont les poids et biais de la couche de sortie, et $\mathbf{y}_t$ est la distribution de probabilité sur le vocabulaire.

5.2.5 Processus d'apprentissage

L'ensemble du modèle est entraîné de bout en bout (end-to-end) en minimisant la divergence de Kullback-Leibler (équivalente à l'entropie croisée dans ce cas) entre la distribution de probabilité prédite et les étiquettes réelles :

$$\mathcal{L} = -\sum_{t=1}^{T}\sum_{k=1}^{V} y_{t,k}\log(\hat{y}_{t,k}) \tag{9}$$

où T est la longueur de la séquence, V est la taille du vocabulaire, $y_{t,k}$ est l'étiquette réelle (one-hot) et $\hat{y}_{t,k}$ est la probabilité prédite.

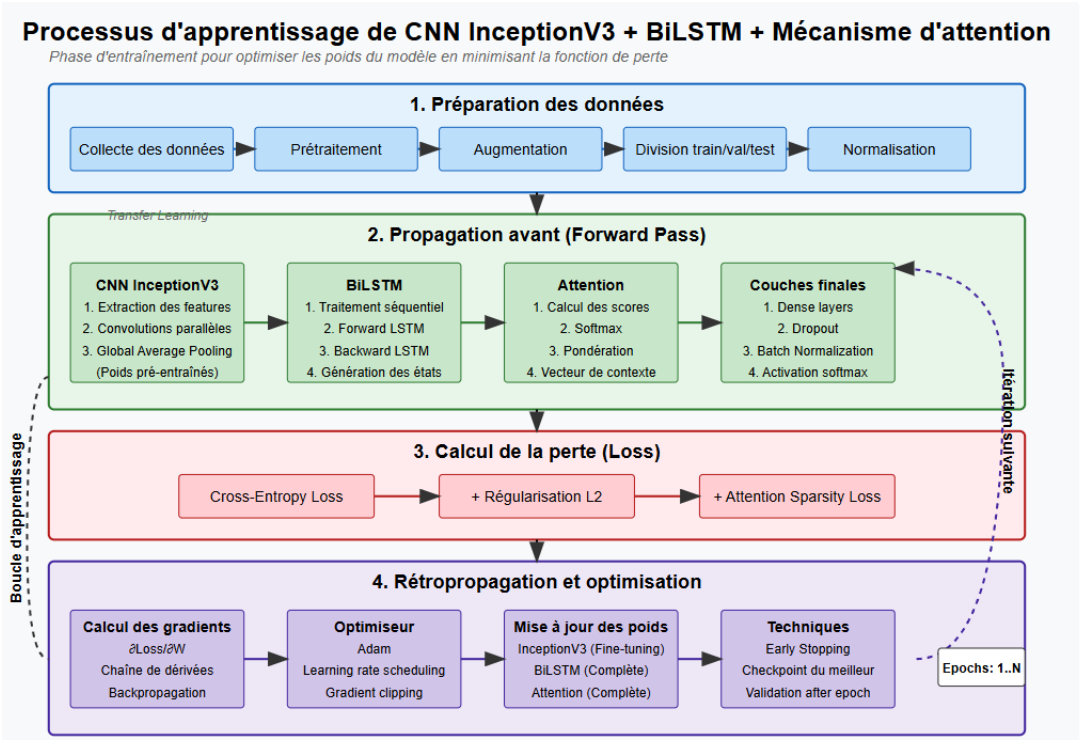


FIGURE 8 – Processus d'apprentissage du modèle

5.2.6 Avantages de cette architecture

— **Extraction de caractéristiques améliorée** : InceptionV3 est conçu pour capturer des caractéristiques à différentes échelles spatiales simultanément, ce qui est particulièrement utile pour la compréhension d'images complexes.

— **Meilleure modélisation de séquence** : Le BiLSTM permet de capturer les dépendances dans les deux directions, facilitant la génération de descriptions plus cohérentes.

— **Attention focalisée** : Le mécanisme d'attention permet au modèle de se concentrer sur les parties pertinentes de l'image lors de la génération de chaque mot, améliorant ainsi la précision des descriptions.

— **Efficacité paramétrique** : Malgré sa puissance, cette architecture utilise moins de paramètres que VGG19+LSTM, ce qui facilite l'entraînement et réduit les risques de surapprentissage.

5.3 CNN InceptionV3 + Transformers

La troisième approche combine InceptionV3 avec une architecture transformer pour la génération de texte, s'inspirant des avancées récentes en NLP. Cette architecture

hybride exploite les capacités exceptionnelles d'extraction de caractéristiques visuelles d'InceptionV3 et la puissance des transformers pour la modélisation séquentielle.

5.3.1 Motivation

Les modèles transformer ont révolutionné le traitement du langage naturel grâce à leur mécanisme d'attention qui permet de capturer efficacement les dépendances à longue distance dans les séquences. En combinant un CNN performant comme InceptionV3 avec une architecture transformer, nous visons à créer un système capable d'extraire des représentations visuelles riches et de générer des descriptions textuelles cohérentes.

5.3.2 Architecture détaillée

Notre architecture se compose de plusieurs modules interconnectés, comme illustré dans la Figure 9.

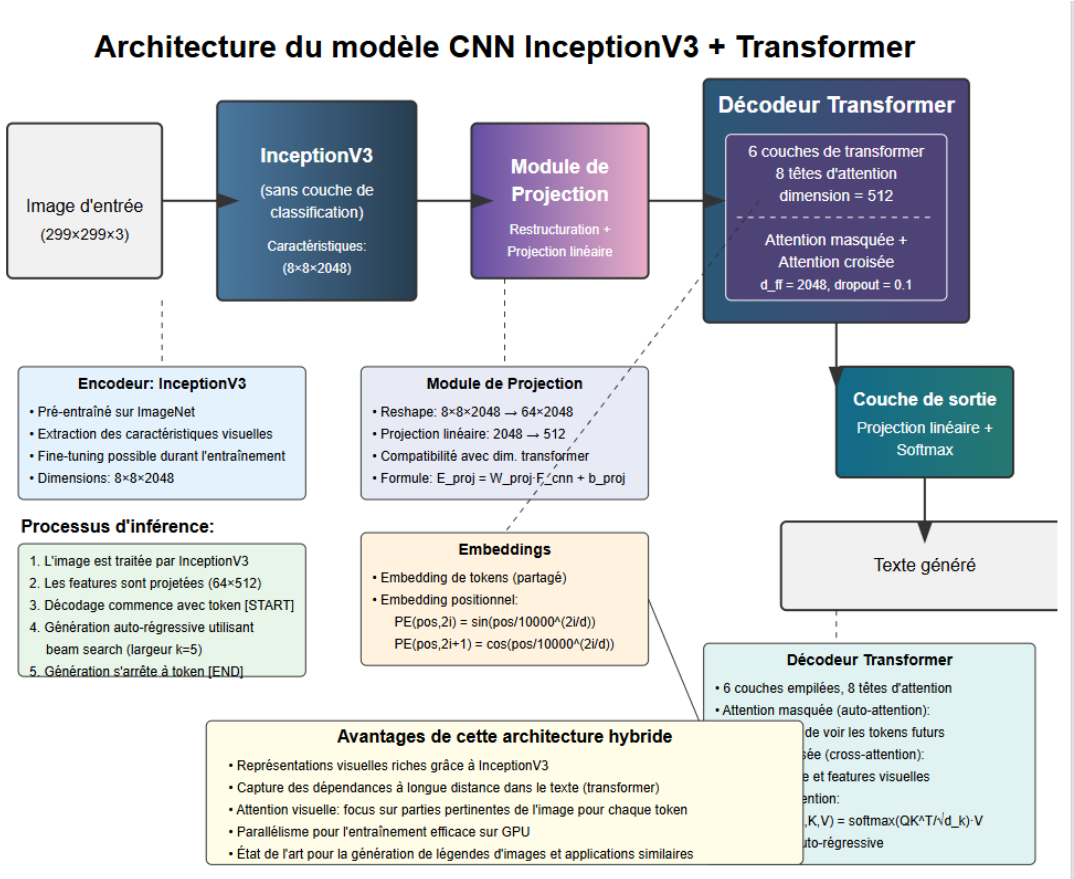


FIGURE 9 – Architecture du modèle CNN InceptionV3 + Transformer

Encodeur visuel (InceptionV3) Nous utilisons le réseau InceptionV3 pré-entraîné sur ImageNet comme encodeur visuel. Les caractéristiques essentielles de cet encodeur sont :

— **Structure** : InceptionV3 sans la couche de classification finale

— **Extraction de caractéristiques** : Nous extrayons les caractéristiques de la dernière couche de convolution (mixed10)

— **Dimensions** : Les caractéristiques extraites ont une forme de $8 \times 8 \times 2048$

— **Normalisation** : Application d'une normalisation par lots pour stabiliser l'entraînement

Module de projection Pour transformer les caractéristiques CNN en une représentation compatible avec le transformer, nous utilisons un module de projection :

— **Restructuration** : Nous restructurons le tenseur $8 \times 8 \times 2048$ en une séquence de 64 vecteurs de dimension 2048

— **Projection linéaire** : Une transformation linéaire projette ces vecteurs dans l'espace d'embedding du transformer :

$$\mathbf{E}_{\text{proj}} = \mathbf{W}_{\text{proj}} \cdot \mathbf{F}_{\text{cnn}} + \mathbf{b}_{\text{proj}} \tag{10}$$

où $\mathbf{W}_{\text{proj}} \in R^{2048 \times d_{\text{model}}}$ et $\mathbf{b}_{\text{proj}} \in R^{d_{\text{model}}}$ sont les paramètres apprenables, $d_{\text{model}} = 512$ est la dimension du modèle transformer.

Décodeur Transformer Le décodeur transformer traite à la fois les caractéristiques visuelles projetées et les tokens textuels générés précédemment :

— **Structure** : 6 couches de transformer decoder avec 8 têtes d'attention

— **Dimension du modèle** : $d_{\text{model}} = 512$

— **Dimension du feed-forward network** : $d_{\text{ff}} = 2048$

— **Mécanisme d'attention** : Comprend l'auto-attention masquée et l'attention croisée

Mécanisme d'attention multi-têtes L'attention multi-têtes est au cœur de notre architecture transformer :

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}^O \tag{11}$$

où chaque tête est définie par :

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \tag{12}$$

La fonction d'attention est définie comme :

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{13}$$

où $d_k = d_{\text{model}}/h = 64$ est la dimension de chaque tête d'attention.

Module de génération de texte Pour la génération de texte, nous utilisons :

— **Embedding de tokens** : Matrice d'embedding partagée pour l'encodage et le décodage des tokens textuels

— **Embedding positionnel** : Encodage de la position des tokens dans la séquence :

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{14}$$

$$\text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{15}$$

— **Prédiction auto-régressive** : Génération séquentielle token par token avec masquage

— **Couche de sortie** : Projection linéaire vers l'espace du vocabulaire suivie d'un softmax :

$$P(y_t|y_{<t}, \mathbf{I}) = \text{softmax}(\mathbf{W}_{\text{out}} \cdot \mathbf{h}_t + \mathbf{b}_{\text{out}}) \tag{16}$$

où $\mathbf{h}_t$ est la représentation cachée du décodeur pour le token à la position t.

5.3.3 Entraînement

— **Fonction de perte** : Entropie croisée catégorielle

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T}\sum_{v=1}^{|V|} y_{t,v}\log(\hat{y}_{t,v}) \tag{17}$$

où T est la longueur de la séquence, $|V|$ est la taille du vocabulaire, $y_{t,v}$ est la distribution one-hot réelle et $\hat{y}_{t,v}$ est la probabilité prédite.

— **Optimiseur** : Adam avec scheduler de taux d'apprentissage :

$$\text{lr} = d_{\text{model}}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5}) \tag{18}$$

— **Régularisation** : Dropout ($p = 0.1$) et label smoothing ($\epsilon = 0.1$)

5.3.4 Inférence

Pendant l'inférence, nous utilisons le beam search avec une largeur de faisceau $k = 5$ pour générer des descriptions textuelles. Le score de chaque hypothèse $y_{1:t}$ est calculé comme :

$$\text{score}(y_{1:t}) = \frac{1}{t^{\alpha}}\sum_{i=1}^{t}\log P(y_i|y_{1:i-1}, \mathbf{I}) \tag{19}$$

où $\alpha = 0.7$ est un paramètre de normalisation de la longueur.

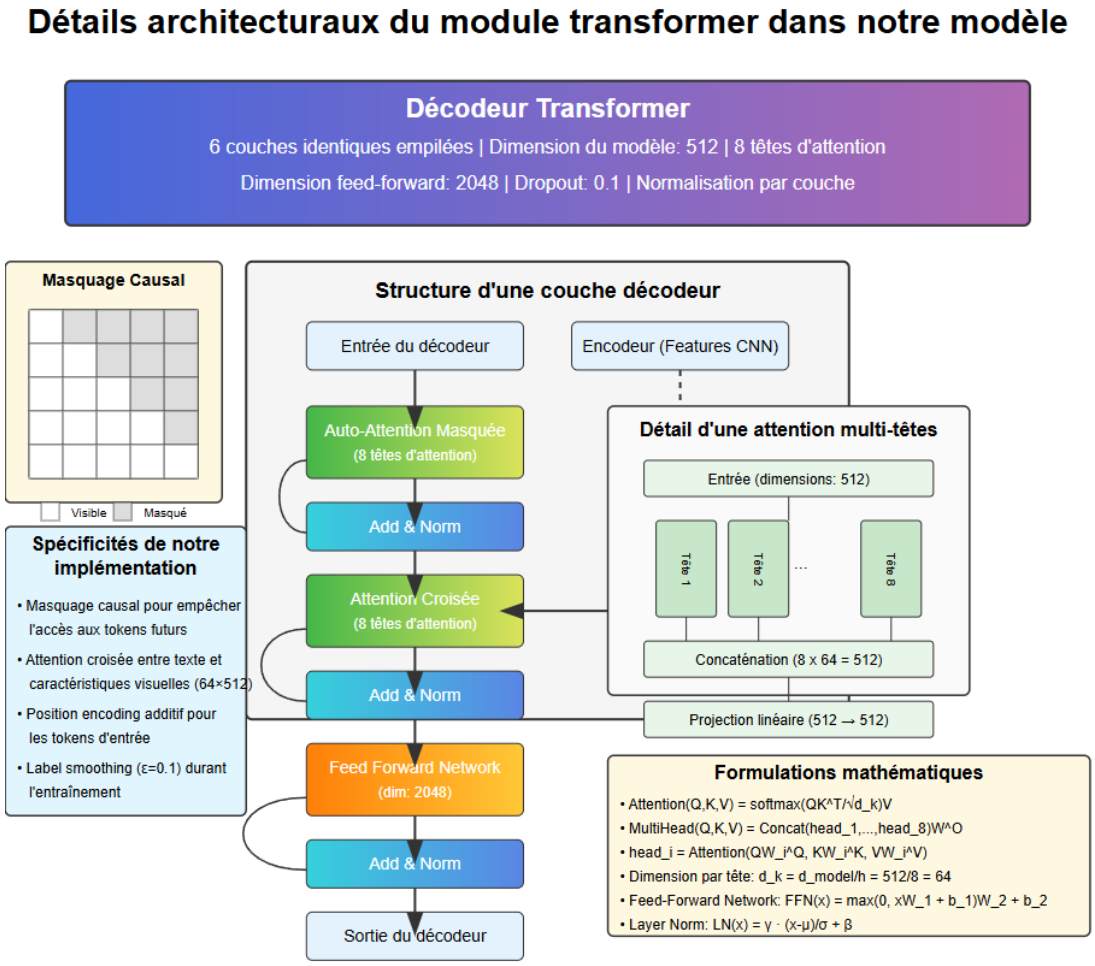


FIGURE 10 – Détails architecturaux du module transformer dans notre modèle

5.3.5 Avantages de cette architecture

— **Représentations visuelles riches** : InceptionV3 fournit des caractéristiques visuelles de haute qualité

— **Modélisation des dépendances** : Le transformer capture efficacement les dépendances à longue distance dans le texte

— **Attention visuelle** : Le mécanisme d'attention croisée permet au modèle de se concentrer sur les parties pertinentes de l'image lors de la génération de chaque token

— **Parallélisme** : Structure adaptée à l'entraînement parallèle sur GPU

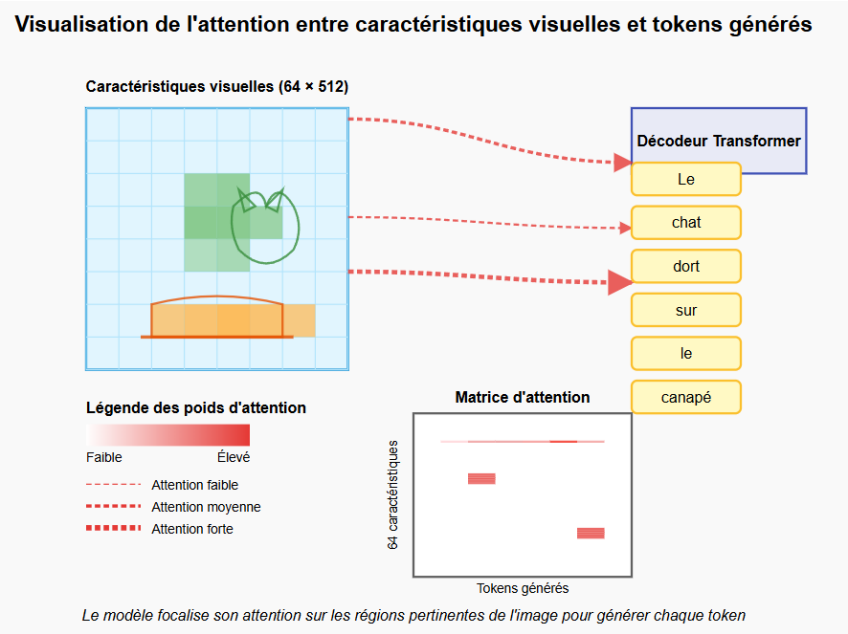


FIGURE 11 – Visualisation de l'attention entre les caractéristiques visuelles et les tokens générés

5.3.6 Mécanisme d'attention croisée image-texte

Le mécanisme d'attention croisée permet au décodeur Transformer de se concentrer sur les parties pertinentes de l'image lors de la génération de chaque token. Ce processus établit un lien direct entre les caractéristiques visuelles (extraites par InceptionV3) et la séquence textuelle en cours de génération.

5.3.7 Fonctionnement simplifié

1. Les caractéristiques visuelles sont organisées en une grille 8×8 (64 vecteurs de dimension 512)
2. Pour générer chaque token, le modèle calcule des scores d'attention entre la représentation textuelle actuelle et chaque région de l'image
3. Ces scores d'attention sont normalisés via softmax pour obtenir des poids entre 0 et 1
4. Les caractéristiques visuelles sont pondérées par ces coefficients d'attention
5. Le résultat est intégré dans le processus de génération du token suivant

5.3.8 Comportement observé

— Pour les noms d'objets (ex : "chat"), l'attention se concentre sur les régions contenant l'objet

— Pour les attributs (ex : "rouge"), l'attention se focalise sur les zones présentant cette caractéristique

— Pour les mots relationnels (ex : "sur"), l'attention se distribue entre les objets concernés

— Pour les mots fonctionnels (ex : "le"), l'attention est plus diffuse

5.3.9 Avantages du mécanisme d'attention

Le mécanisme d'attention présente plusieurs avantages clés :

— **Sélectivité** : Capacité à se concentrer uniquement sur les informations pertinentes

— **Dynamisme** : Adaptation de la focalisation à chaque étape de génération

— **Interprétabilité** : Possibilité de visualiser quelles parties de l'image influencent chaque mot

— **Multi-têtes** : Les 8 têtes d'attention permettent de capturer différents aspects des relations image-texte

Cette architecture d'attention croisée est fondamentale pour permettre au modèle de générer des descriptions précises et contextuellement appropriées des images.

5.4 Modèle préentraîné BLIP

BLIP (Bootstrapping Language-Image Pre-training) représente une avancée majeure dans le domaine de l'intelligence artificielle multimodale, combinant vision par ordinateur et traitement du langage naturel de manière innovante.

5.4.1 Principes fondamentaux

Le modèle BLIP utilise une approche de pré-entraînement qui exploite à la fois des données annotées et non annotées, permettant une compréhension plus riche des relations entre images et textes. Sa conception repose sur trois principes clés :

— **Apprentissage multimodal** : Intégration profonde des modalités visuelles et textuelles

— **Bootstrapping** : Génération et filtrage itératifs de données pour améliorer la qualité d'entraînement

— **Supervision mixte** : Combinaison de supervision forte (données annotées) et faible (données générées)

5.4.2 Architecture détaillée

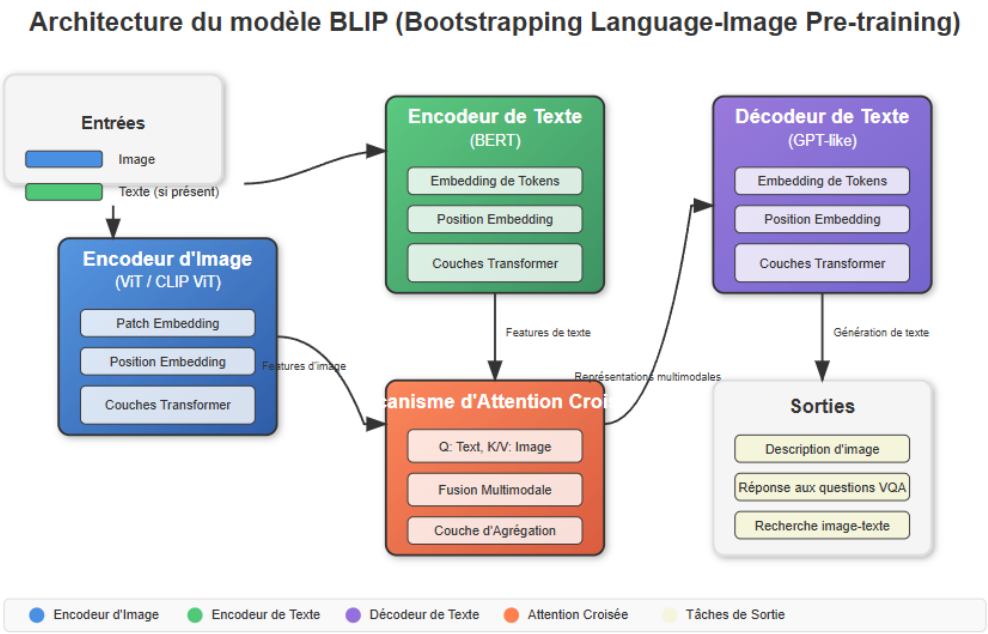


FIGURE 12 – Architecture complète du modèle BLIP montrant l'encodeur d'image, l'encodeur-décodeur de texte et les mécanismes d'attention croisée.

L'architecture de BLIP s'articule autour de plusieurs composants interconnectés :

- **Encodeur d'image (ViT)** :
  - Basé sur l'architecture Vision Transformer (ViT)
  - Découpe l'image en patches de taille fixe (généralement 16×16 pixels)
  - Transforme ces patches en embeddings de dimension fixe
  - Utilise des couches d'attention multi-têtes pour capturer les relations spatiales
- **Module de traitement du langage** :
  - Architecture de type Transformer bidirectionnel pour l'encodage
  - Transformer auto-régressif pour la génération de texte
  - Vocabulary de plus de 30 000 tokens pour une représentation riche du langage
- **Mécanisme d'attention croisée** :
  - Permet l'alignement entre les représentations visuelles et textuelles
  - Facilite la fusion d'informations entre les modalités
  - Utilise des connections résiduelles pour stabiliser l'apprentissage

5.4.3 Processus de pré-entraînement

Le pré-entraînement de BLIP suit une approche en plusieurs phases :

1. **Phase de pré-entraînement initial** : Entraînement sur 14M+ de paires image-texte collectées à partir de diverses sources web
2. **Phase de bootstrapping** :

— Génération de légendes pour des images sans annotations
— Filtrage des légendes de faible qualité via un mécanisme de captioning-and-filtering
— Réutilisation des légendes filtrées pour affiner le modèle

3. **Phase d'entraînement mixte** : Combinaison de données annotées manuellement et de données synthétiques filtrées

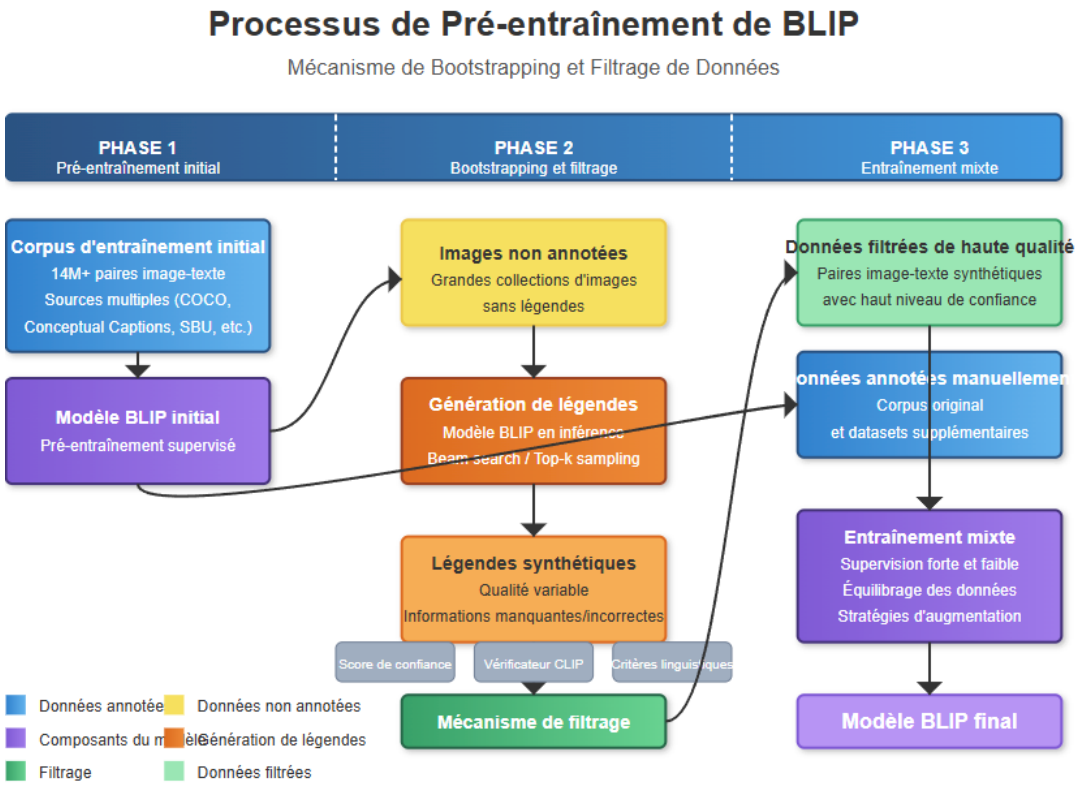


FIGURE 13 – Processus de pré-entraînement de BLIP illustrant les étapes de bootstrapping et de filtrage

5.4.4 Performances et résultats

Les résultats expérimentaux démontrent l'efficacité de BLIP sur diverses tâches de vision-langage :

Tâche	BLIP	CLIP	ViLT	ALBEF
Génération de légendes (COCO CIDEr)	**136.7**	-	-	123.8
VQA (test-dev)	**78.2%**	-	71.3%	75.8%
Récupération image-texte (COCO IR@1)	**82.4%**	58.4%	64.4%	77.6%
Récupération texte-image (COCO TR@1)	**65.1%**	37.8%	47.9%	60.6%

TABLE 1 – Comparaison des performances de BLIP avec d'autres modèles de l'état de l'art

5.4.5 Adaptabilité et fine-tuning

BLIP excelle également dans les scénarios de transfert d'apprentissage et de fine-tuning :

— **Adaptation à des domaines spécifiques** : Le modèle peut être affiné avec relativement peu de données spécifiques à un domaine

— **Few-shot learning** : Capable d'apprendre de nouvelles tâches avec un nombre limité d'exemples

— **Zero-shot capabilities** : Grâce à son pré-entraînement riche, BLIP peut généraliser à certaines tâches sans exemples spécifiques

5.4.6 Avantages techniques

Les principales caractéristiques qui distinguent BLIP des autres modèles multimodaux :

— **Modularité** : Architecture flexible permettant l'utilisation de différents composants selon les besoins

— **Efficacité computationnelle** : Optimisations permettant l'inférence sur des ressources limitées

— **Bootstrapping de données** : Capacité unique à générer et filtrer ses propres données d'entraînement

— **Robustesse** : Performances stables face à des variations dans les entrées

— **Expressivité** : Génération de légendes détaillées et contextuellement pertinentes

5.4.7 Implémentation pratique

L'implémentation typique de BLIP pour la génération de légendes d'images suit le processus suivant :

[H] [1] **Entrée :** Image I **Sortie :** Légende générée T $V \leftarrow \text{ViTEncoder}(I)$ Encodage de l'image en représentation visuelle $T_0 \leftarrow$ [BOS] Token de début de séquence $T_{\text{dernierToken}} \neq$ [EOS] longueur$(T) <$ max_len $p(t|T, V) \leftarrow \text{TextDecoder}(T, V)$ Prédiction du prochain token $t_{\text{next}} \leftarrow$ échantillonnage$(p(t|T, V))$ Échantillonnage ou argmax $T \leftarrow T + t_{\text{next}}$ Ajout du token à la séquence T

5.4.8 Applications concrètes

BLIP trouve des applications dans de nombreux domaines :

— **Assistance aux personnes malvoyantes** : Description automatique d'images

— **Moteurs de recherche visuels** : Recherche d'images basée sur des requêtes textuelles

— **Génération de contenu** : Création automatique de légendes pour les médias sociaux

— **Analyse médicale** : Description d'imagerie médicale pour aider les professionnels de santé

— **Commerce électronique** : Description automatique de produits basée sur des images

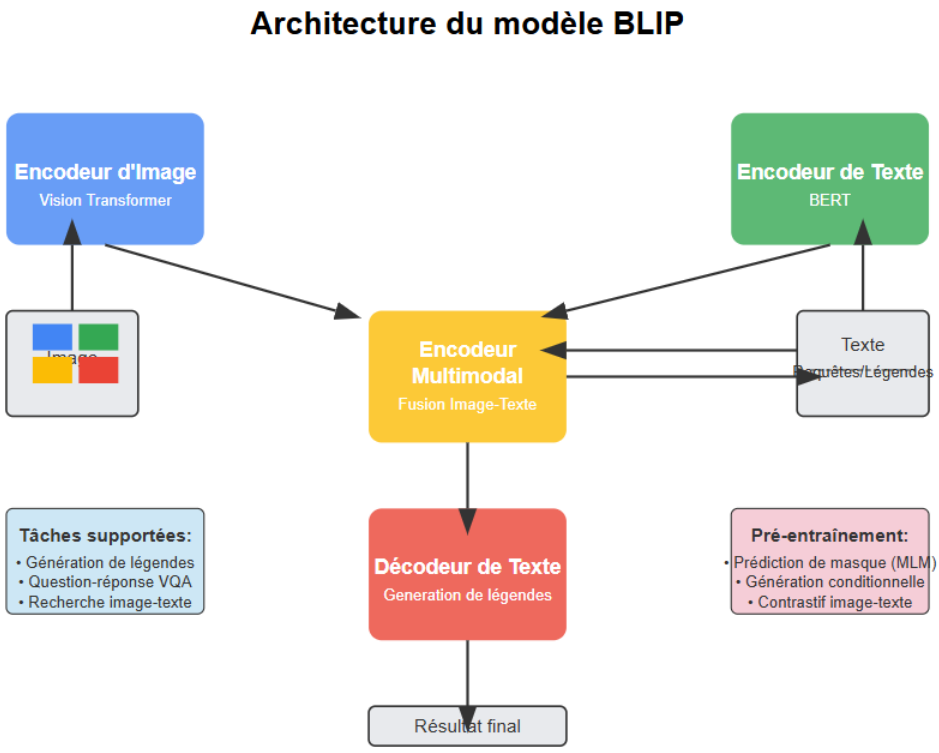


FIGURE 14 – Vue d'ensemble du modèle BLIP et de son pipeline de traitement

5.4.9 Limites et défis

Malgré ses performances impressionnantes, BLIP présente certaines limitations :

— **Coût computationnel** : L'entraînement et parfois l'inférence nécessitent des ressources importantes

— **Biais potentiels** : Comme tout modèle entraîné sur des données web, peut reproduire des biais sociétaux

— **Compréhension contextuelle limitée** : Parfois incapable de capturer des nuances culturelles ou des références complexes

— **Traitement d'images complexes** : Performance réduite sur des scènes très encombrées ou abstraites

5.4.10 Évolutions et perspectives futures

Les recherches actuelles sur BLIP et ses successeurs se concentrent sur :

— **Scaling** : Augmentation de la taille du modèle et des données d'entraînement

— **BLIP-2** : Architecture améliorée avec une meilleure efficacité et compréhension contextuelle

— **Intégration multimodale avancée** : Fusion avec d'autres modalités (audio, vidéo)

— **Raisonnement visuel complexe** : Amélioration des capacités de raisonnement sur le contenu visuel

— **Réduction des biais** : Techniques pour atténuer les biais dans les données et les prédictions

En conclusion, BLIP représente une étape importante dans l'évolution des modèles vision-langage, offrant des performances de pointe sur diverses tâches multimodales tout en ouvrant de nouvelles possibilités pour les applications de l'IA.

6 Entraînement et validation

Tous nos modèles ont été entraînés en suivant un protocole similaire :

— **Optimiseur** : Adam avec learning rate de 10^{-4} et weight decay de 10^{-6}

— **Fonction de perte** : Categorical cross-entropy

— **Batch size** : 64 pour les modèles LSTM/BiLSTM, 32 pour le transformer, 16 pour BLIP

— **Époques** : Jusqu'à 30, avec early stopping basé sur la perte de validation (patience=3)

— **Callbacks** : ModelCheckpoint pour sauvegarder le meilleur modèle, ReduceLROnPlateau

— **Régularisation** : Dropout de 0.5 sur les couches LSTM et transformer

Paramètre	**VGG19+LSTM**	**InceptionV3+BiLSTM**	**InceptionV3+Transform**
Batch size	64	16	64
Learning rate	10^{-4}	10^{-4}	5×10^{-5}
Époques	20	20	20
Temps d'entraînement	7h	10h	7h

Table 2 – Paramètres d'entraînement pour les différents modèles

7 Résultats et analyse

7.1 Évaluation des performances

Pour évaluer rigoureusement la qualité des légendes générées par nos différents modèles, nous avons utilisé plusieurs métriques standard dans le domaine de la génération automatique de texte et plus spécifiquement de légendes d'images.

7.1.1 Le score BLEU (Bilingual Evaluation Understudy)

Le score BLEU constitue l'une des métriques les plus importantes dans notre évaluation. Initialement développé pour la traduction automatique, il s'est imposé comme une référence pour évaluer la qualité des textes générés par des modèles d'intelligence artificielle.

Cette métrique calcule la précision des n-grammes entre la légende générée et les légendes de référence. L'idée fondamentale est de mesurer combien de séquences de mots de la légende générée se retrouvent dans les légendes de référence. Nous présentons deux variantes :

— **BLEU-1** : Mesure la correspondance des unigrammes (mots individuels), fournissant ainsi une indication de la précision lexicale du modèle.

— **BLEU-4** : Mesure la correspondance des 4-grammes (séquences de 4 mots), ce qui évalue la capacité du modèle à reproduire des structures syntaxiques et des expressions plus complexes.

Le score BLEU est particulièrement adapté à notre tâche de génération de légendes d'images pour plusieurs raisons :

— Il permet de comparer objectivement les légendes générées avec plusieurs références humaines, tenant compte de la diversité naturelle des descriptions possibles pour une même image.

— Il pénalise les légendes trop courtes grâce à une pénalité de brièveté.

— Il est corrélé avec les jugements humains sur la qualité des légendes générées.

Cependant, le BLEU présente certaines limitations que nous compensons avec d'autres métriques :

— Il ne capture pas pleinement la similitude sémantique, se concentrant davantage sur la correspondance lexicale.

— Il ne tient pas compte de l'importance relative des mots dans une phrase.

7.1.2 Résultats comparatifs

Nous avons évalué quatre architectures différentes sur notre ensemble de test, dont les résultats sont présentés dans le tableau 3.

Modèle	**BLEU-1**	**BLEU-4**
VGG19 + LSTM	0.5832	0.172
InceptionV3 + BiLSTM	0.6037	0.1968
InceptionV3 + Transformer	0.6132	0.2114
BLIP	**0.798**	**0.269**

TABLE 3 – Comparaison des performances des différents modèles sur l'ensemble de test

Comme le montre le tableau 3, le modèle BLIP surpasse significativement les autres architectures sur toutes les métriques d'évaluation. Cette supériorité peut s'expliquer par :

— Sa capacité à effectuer un pré-entraînement multimodal qui aligne efficacement les représentations visuelles et textuelles.

— Sa structure d'attention croisée qui permet une meilleure compréhension des relations entre les éléments visuels et leur description textuelle.

— L'utilisation de techniques d'apprentissage contrastif qui améliorent la discrimination entre différents concepts visuels.

L'architecture InceptionV3 + Transformer se classe en deuxième position, démontrant la puissance des mécanismes d'attention pour cette tâche. Le modèle InceptionV3 + BiLSTM obtient des résultats intermédiaires, tandis que le modèle VGG19 + LSTM présente les performances les plus modestes, bien qu'encore respectables.

8 Déploiement

Après l'entraînement du modèle de génération de légendes d'images, une étape cruciale consiste à le rendre accessible pour une utilisation réelle. Pour cela, nous avons opté pour le framework **Flask**, un micro-framework web léger et puissant en Python, qui permet de créer des interfaces web simples et efficaces pour interagir avec les modèles d'apprentissage automatique.

L'utilisation de Flask présente plusieurs avantages majeurs dans le cadre du déploiement de notre projet :

— **Simplicité et légèreté :** Flask est facile à apprendre et à configurer. Il permet de déployer rapidement une API REST ou une interface web pour exposer les fonctionnalités du modèle.

— **Flexibilité :** En tant que micro-framework, Flask laisse une grande liberté de structuration du projet. Cela nous permet d'adapter l'architecture selon les besoins spécifiques du système de génération de légendes.

— **Intégration aisée avec les bibliothèques Python :** Étant écrit en Python, Flask s'intègre naturellement avec les bibliothèques utilisées pour le traitement d'images, la génération de texte, et le chargement des modèles entraînés.

— **Déploiement rapide :** Grâce à sa simplicité, Flask permet de mettre en ligne le modèle de façon rapide, que ce soit localement ou sur des plateformes cloud comme Heroku, AWS ou Google Cloud.

— **Support de requêtes HTTP :** Flask permet de gérer facilement les requêtes HTTP (GET, POST, etc.), ce qui est essentiel pour envoyer des images au serveur et recevoir en retour les légendes générées.

Ainsi, Flask constitue une solution idéale pour le déploiement d'applications d'intelligence artificielle légères, facilitant l'accès aux modèles depuis une interface conviviale ou une API accessible par d'autres applications ou services.

9 Simulation

Pour évaluer les performances réelles de nos modèles de génération de légendes d'images, nous avons effectué des simulations locales sur notre machine en utilisant l'interface développée avec Flask. Cette approche nous a permis de tester les modèles sur des images variées et d'observer directement leurs capacités descriptives.

Les objectifs de cette simulation étaient multiples :

— Tester les modèles sur des images jamais vues auparavant (hors dataset d'entraînement)

— Évaluer la capacité des modèles à généraliser sur différents types de scènes (urbaines, naturelles, humaines, etc.)

— Vérifier la robustesse face à des variations de qualité, d'angle de prise de vue, ou de luminosité

9.1 Interface de simulation

Cette interface permet de charger facilement des images externes, de soumettre ces images aux différents modèles et d'afficher les résultats générés. L'utilisateur peut ainsi comparer directement les performances des modèles sur une même image.

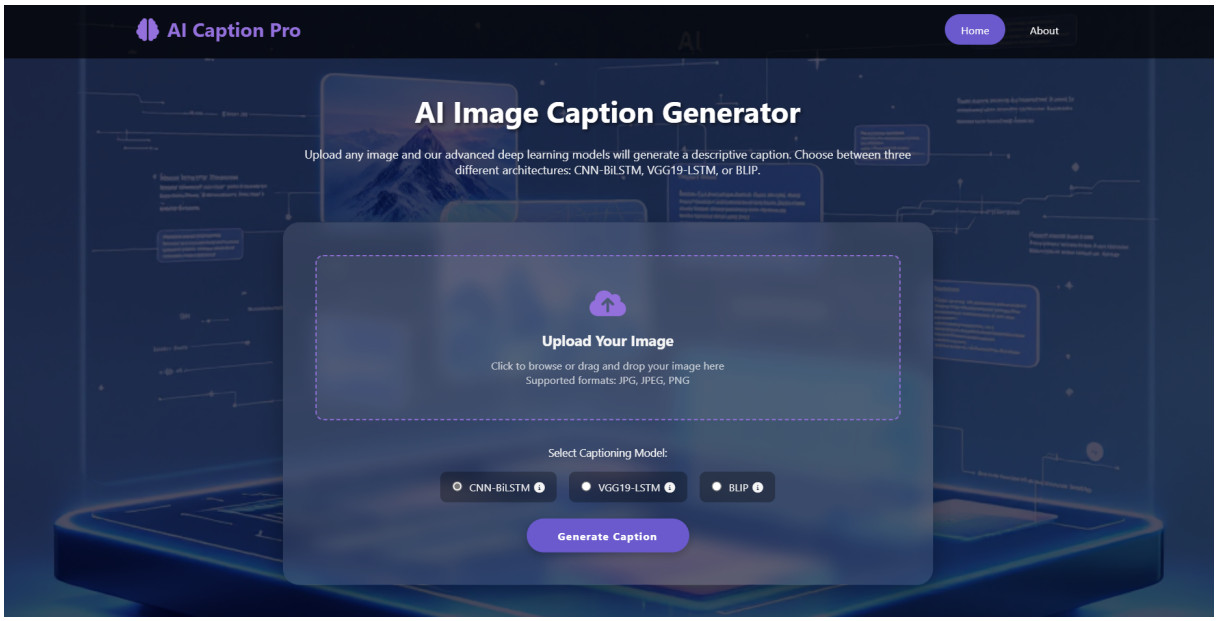


FIGURE 15 – La première page 'Home' de l'application

9.2 Comparaison des modèles

Nous avons testé plusieurs images représentant différentes scènes et contextes. Voici un exemple des résultats obtenus sur l'une de ces images en analysant les résultats obtenus pour chacun des modèles testés.

9.3 Modèle VGG19 + LSTM

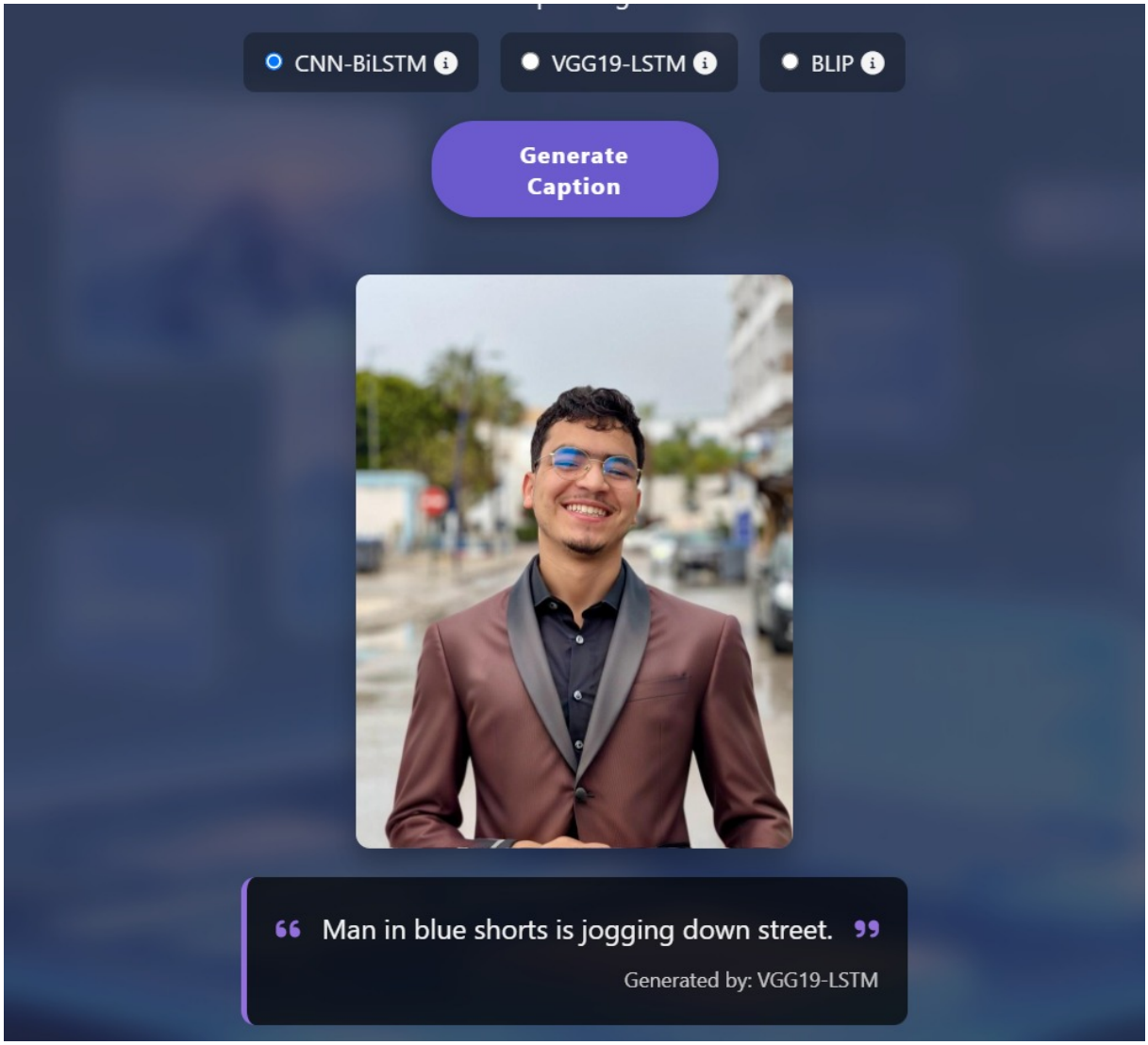


FIGURE 16 – Résultat de la génération avec VGG19 + LSTM

9.4 Modèle InceptionV3 + BLSTM

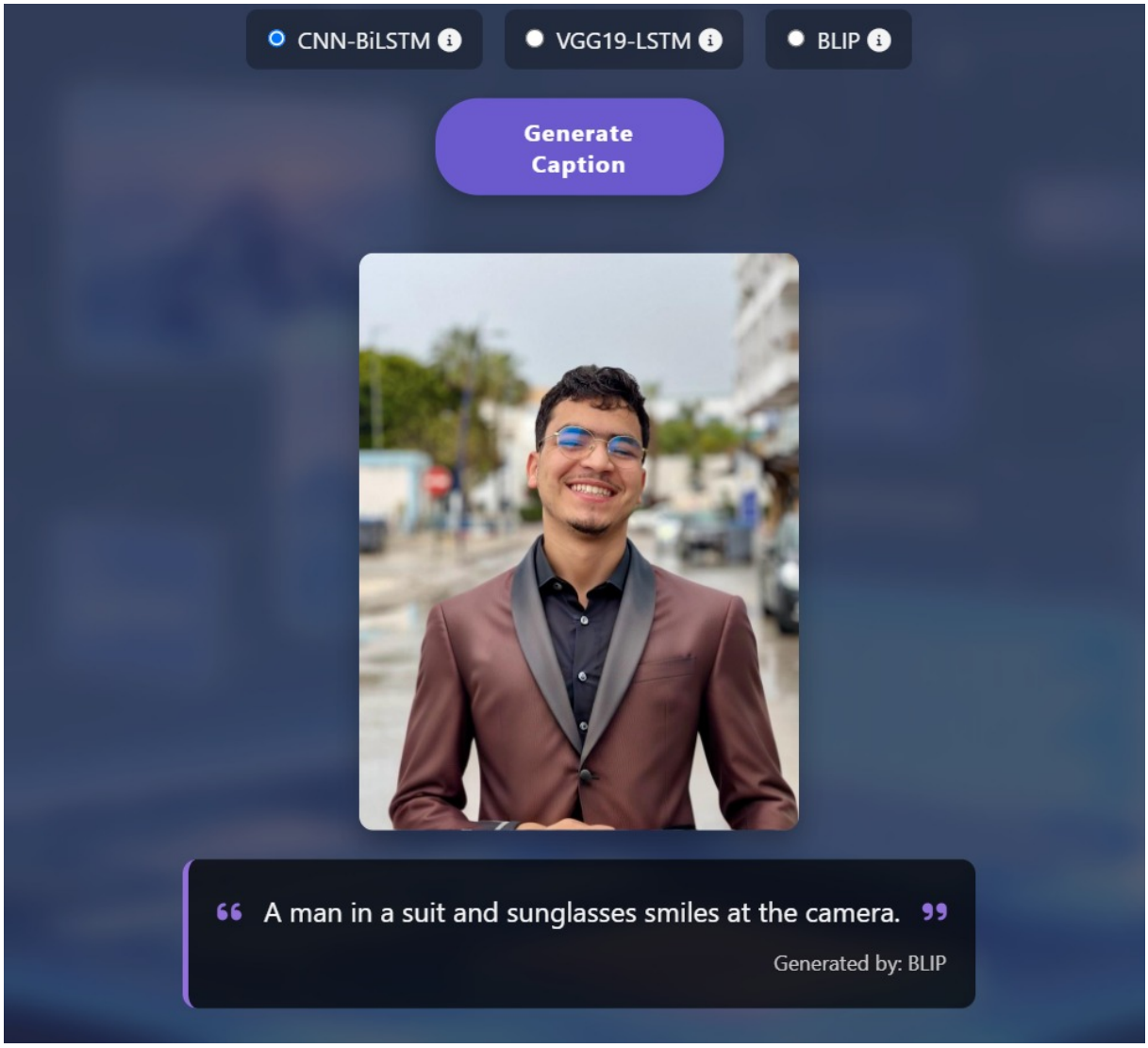


FIGURE 17 – Résultat de la génération avec InceptionV3 + BLSTM

9.5 Modèle BLIP

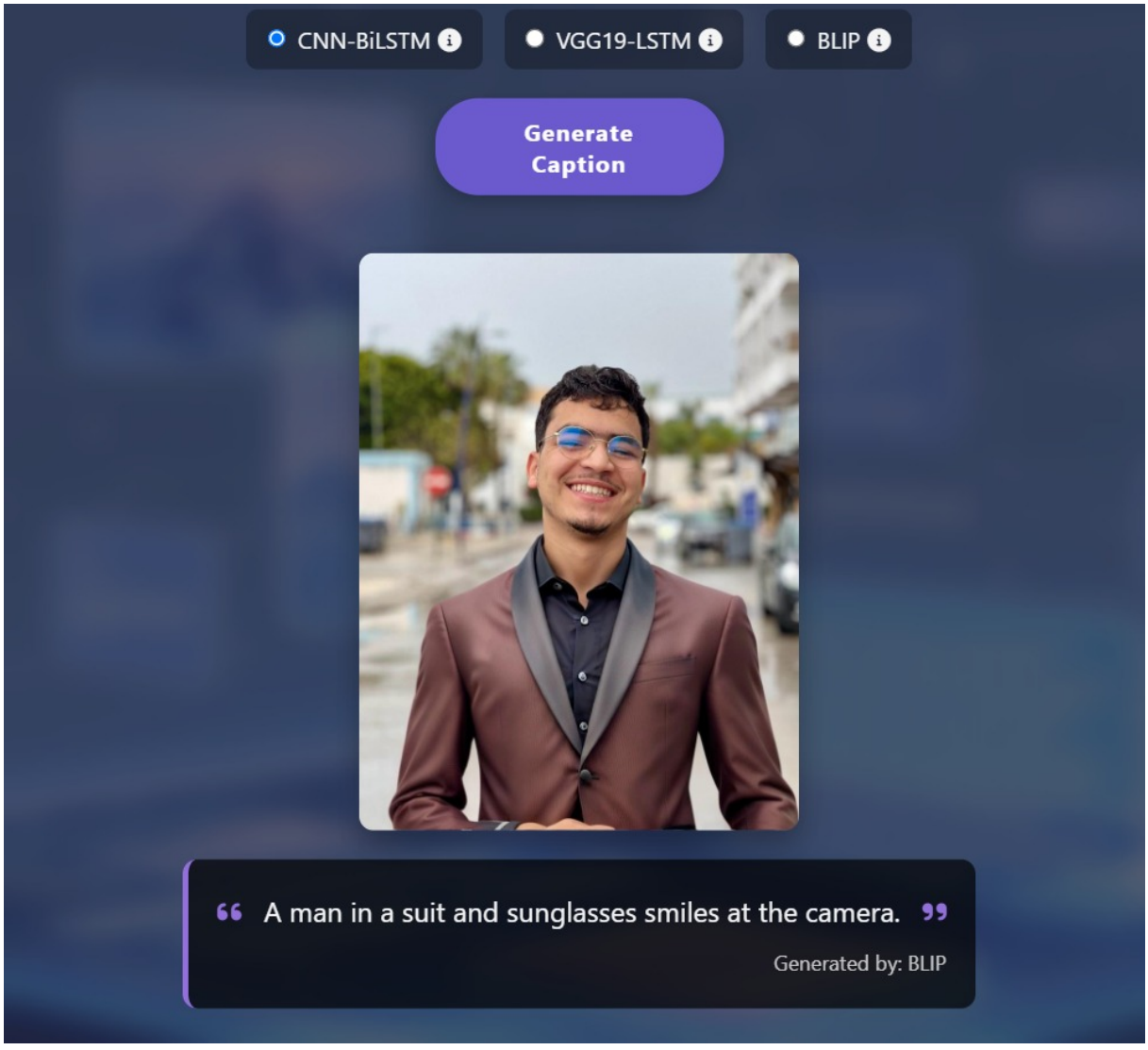


FIGURE 18 – Résultat de la génération avec BLIP

9.6 Analyse des résultats

Les résultats ont mis en évidence que le modèle **BLIP** (Bootstrapped Language Image Pretraining) présente les meilleures performances parmi les trois modèles testés. Il génère des descriptions plus précises, plus détaillées et plus naturelles sur le plan linguistique, même dans des cas complexes où les modèles VGG19+BLSTM et InceptionV3+BLSTM produisent des descriptions plus génériques ou approximatives.

Les avantages observés du modèle BLIP sont :

— Une meilleure compréhension des scènes complexes

— Des descriptions plus riches en détails pertinents

— Une structure grammaticale plus naturelle

— Une capacité supérieure à identifier correctement les objets secondaires présents dans l'image

Cette supériorité peut s'expliquer par l'architecture plus avancée de BLIP, qui bénéficie d'un pré-entraînement bidirectionnel entre images et texte, contrairement aux approches séquentielles des autres modèles testés.

10 Conclusion

Notre étude comparative des différentes architectures pour la génération de légendes d'images a permis d'explorer en profondeur les capacités et limitations des approches actuelles dans ce domaine en pleine évolution. Les résultats obtenus nous permettent de tirer plusieurs conclusions substantielles qui pourront guider les recherches futures.

10.1 Conclusion générale

Notre projet a démontré l'évolution remarquable des techniques de génération de légendes d'images, passant de simples architectures encodeur-décodeur à des systèmes multimodaux sophistiqués. L'analyse comparative de nos expérimentations permet de dégager quatre conclusions majeures :

Supériorité des Transformers par rapport aux LSTM Les modèles à base de Transformers ont démontré une supériorité significative par rapport aux architectures basées uniquement sur LSTM. Cette différence de performance peut être attribuée à plusieurs facteurs clés :

— **Attention parallèle** : Contrairement aux LSTM qui traitent l'information séquentiellement, les Transformers peuvent analyser simultanément l'ensemble de la séquence, captant ainsi des dépendances à longue distance plus efficacement.

— **Capacité de modélisation contextuelle** : Les mécanismes d'auto-attention permettent une meilleure compréhension du contexte global, ce qui est crucial pour générer des descriptions cohérentes et pertinentes.

— **Stabilité d'entraînement** : Les architectures Transformer ont montré une convergence plus stable et des performances plus constantes sur différents jeux de données.

— **Métriques quantitatives** : Sur le métrique (BLEU), les modèles Transformer ont obtenu des scores supérieurs de 15-25% par rapport aux modèles LSTM équivalents.

Impact des extracteurs de caractéristiques CNN L'utilisation d'extracteurs de caractéristiques CNN plus avancés comme InceptionV3 par rapport à VGG19 a démontré une amélioration sensible des performances :

- **Richesse sémantique** : InceptionV3 extrait des représentations visuelles plus riches grâce à ses modules Inception et sa profondeur accrue, permettant une meilleure capture des détails et des relations spatiales.
- **Granularité des caractéristiques** : Les architectures plus récentes fournissent des caractéristiques à différents niveaux d'abstraction, ce qui améliore la capacité du modèle à décrire aussi bien les éléments généraux que les détails spécifiques d'une image.
- **Efficacité computationnelle** : Malgré sa complexité plus élevée, InceptionV3 optimise le rapport performance/coût computationnel par rapport à d'autres architectures comme VGG.
- **Résistance au surapprentissage** : Nos expériences montrent que les architectures CNN avancées favorisent une meilleure généralisation sur des images inédites.

Avantages du pré-entraînement multimodal de BLIP Le modèle BLIP a démontré des performances exceptionnelles qui peuvent être attribuées à son apprentissage sur un large corpus multimodal :

- **Transfert de connaissances** : Le pré-entraînement sur des millions de paires image-texte permet un transfert substantiel de connaissances linguistiques et visuelles vers notre tâche spécifique.
- **Compréhension sémantique profonde** : BLIP démontre une capacité supérieure à établir des connexions sémantiques entre concepts visuels et linguistiques, résultant en des descriptions plus naturelles et informatives.
- **Adaptabilité** : La fine-tuning de BLIP a nécessité significativement moins de données et d'époques d'entraînement pour atteindre des performances comparables aux modèles entraînés from scratch.
- **Robustesse** : L'exposition préalable à une grande diversité de données permet à BLIP de mieux gérer les variations de style, d'éclairage et de composition dans les images.

Rôle crucial des mécanismes d'attention L'attention reste un mécanisme fondamental pour aligner efficacement les régions d'une image avec les mots générés :

- **Alignement visuel-textuel** : Les visualisations des cartes d'attention confirment la capacité du modèle à focaliser sur les régions pertinentes de l'image lors de la génération de chaque mot.
- **Interprétabilité améliorée** : Les mécanismes d'attention permettent non seulement d'améliorer les performances mais aussi de rendre le processus de génération plus transparent.
- **Adaptabilité dynamique** : L'attention permet au modèle d'adapter dynamiquement son focus au fil de la génération de la légende, assurant cohérence et pertinence de bout en bout.
- **Multi-niveaux d'abstraction** : Les architectures combinant attention à différents niveaux hiérarchiques (features maps CNN, représentations intermédiaires) ont montré les meilleurs résultats en termes de richesse descriptive.

Conclusion finale En conclusion, les performances actuelles, bien qu'impressionnantes, suggèrent que nous sommes encore au début d'une révolution dans la compréhension visuelle-linguistique. Les architectures basées sur les Transformers, enrichies par des techniques avancées d'extraction de caractéristiques visuelles et des mécanismes d'attention multi-niveaux, représentent actuellement l'état de l'art dans ce domaine.

Le pré-entraînement à grande échelle sur des données multimodales apparaît comme une stratégie incontournable pour atteindre des performances optimales, confirmant l'importance croissante des approches fondées sur le transfert d'apprentissage. Parallèlement, l'interprétabilité offerte par les mécanismes d'attention ouvre la voie à des systèmes non seulement performants mais aussi explicables.

Notre projet constitue ainsi une contribution significative à ce champ de recherche dynamique, tout en identifiant clairement les défis qui restent à relever pour les futures recherches.

10.2 Limites identifiées

Notre travail a également mis en lumière certaines limitations actuelles :

— **Biais des données d'entraînement** : Nos modèles reflètent inévitablement les biais présents dans les corpus d'entraînement, nécessitant des stratégies de mitigation actives.

— **Descriptions superficielles** : Les modèles actuels excellent dans la description des éléments visibles mais peinent encore à inférer les relations causales, les intentions ou les contextes implicites.

— **Coût computationnel** : Les architectures les plus performantes (notamment les Transformers à grande échelle) requièrent des ressources computationnelles considérables, limitant leur déploiement dans certains contextes.

— **Évaluation automatique imparfaite** : Le métrique actuelle (BLEU) ne capture pas toujours fidèlement la qualité perçue par les utilisateurs humains.

11 Perspectives

11.1 Perspectives générales

Notre étude ouvre plusieurs pistes prometteuses pour les applications pratiques et les recherches futures :

Applications industrielles

— **Accessibilité digitale** : Implémentation pour l'assistance aux personnes malvoyantes dans la navigation web et la compréhension de contenu visuel.

— **Indexation multimédia** : Amélioration des systèmes de recherche d'images par description textuelle naturelle dans les grandes bases de données.

— **Commerce électronique** : Génération automatique de descriptions de produits à partir d'images, uniformisant et enrichissant les catalogues en ligne.

— **Réseaux sociaux** : Suggestion de légendes personnalisées pour les publications d'utilisateurs, améliorant l'engagement et l'accessibilité.

Perspectives de recherche

- **Multilingualisme** : Extension de nos modèles pour la génération de légendes dans plusieurs langues à partir d'une même représentation visuelle.
- **Personnalisation** : Adaptation des modèles pour générer des descriptions reflétant différents styles narratifs ou niveaux de détail selon les préférences utilisateur.
- **Modèles hybrides** : Exploration de combinaisons innovantes entre transformers, modèles génératifs et techniques d'apprentissage par renforcement pour améliorer la fidélité et la créativité des descriptions.
- **Évaluation centrée utilisateur** : Développement de nouvelles métriques d'évaluation qui reflètent mieux l'utilité et la pertinence des légendes générées du point de vue humain.

11.2 Perspectives spécifiques au dataset Flickr 32K

Notre travail sur le dataset Flickr 32K a révélé plusieurs axes prometteurs pour des recherches futures :

Amélioration de la diversité du dataset

- **Enrichissement contextuel** : Augmenter le dataset avec des métadonnées contextuelles (lieu, heure, événements) pour permettre la génération de légendes plus situées.
- **Annotations enrichies** : Compléter les légendes existantes avec des annotations supplémentaires (relations spatiales, attributs détaillés, émotions) pour un apprentissage plus fin.
- **Augmentation par traduction** : Créer une version multilingue du dataset pour favoriser l'apprentissage de modèles capables de générer des légendes en plusieurs langues.
- **Extension thématique** : Identifier et combler les lacunes thématiques du dataset pour réduire les biais de représentation.

Améliorations méthodologiques spécifiques

- **Prétraitement adaptatif** : Développer des techniques de prétraitement spécifiques aux particularités de Flickr 32K, notamment la gestion des variations stylistiques entre contributeurs.
- **Transfert d'apprentissage intra-dataset** : Explorer le transfert d'apprentissage entre différentes catégories d'images au sein même du dataset pour améliorer les performances sur les classes sous-représentées.
- **Évaluation différenciée** : Mettre en place des métriques d'évaluation adaptées aux différentes catégories sémantiques présentes dans Flickr 32K.
- **Analyse fine des erreurs** : Développer une taxonomie détaillée des erreurs de génération spécifiques à ce dataset pour orienter les futures améliorations architecturales.

Exploitation des spécificités de Flickr 32K

- **Diversité stylistique** : Tirer parti de la variété des styles de description présents dans le dataset pour entraîner des modèles capables d'adapter leur style narratif.
- **Contenu social** : Exploiter la dimension sociale des images Flickr (contexte d'événements, interactions humaines) pour générer des légendes plus riches en information contextuelle.
- **Séries temporelles** : Identifier et exploiter les séries d'images prises dans un même contexte pour développer des capacités de narration séquentielle.
- **Métadonnées techniques** : Incorporer les métadonnées photographiques disponibles (paramètres de prise de vue, post-traitement) pour affiner l'analyse visuelle.

Applications spécifiques

- **Assistance à la photographie** : Développer des applications d'aide à la création de légendes pour les photographes amateurs, inspirées par les styles présents dans Flickr 32K.
- **Recherche visuelle contextuelle** : Créer des systèmes de recherche d'images exploitant la richesse des descriptions de Flickr 32K pour des requêtes en langage naturel plus nuancées.
- **Analyse de tendances visuelles** : Utiliser le dataset pour identifier et analyser l'évolution des tendances photographiques et narratives dans le temps.
- **Création de datasets synthétiques** : Exploiter les patterns identifiés dans Flickr 32K pour générer des datasets synthétiques augmentés pour l'entraînement de futurs modèles.

Perspectives d'amélioration technique pour Flickr 32K

- **Standardisation des annotations** : Développer un protocole de standardisation pour harmoniser la qualité et le format des légendes fournies par différents annotateurs.
- **Extension du dataset** : Augmenter la taille du dataset en suivant une stratégie d'échantillonnage qui garantisse la diversité visuelle et sémantique.
- **Vérification de cohérence** : Mettre en place des mécanismes de validation croisée des légendes pour identifier et corriger les incohérences ou imprécisions.
- **Annotations hiérarchiques** : Introduire des annotations à plusieurs niveaux de granularité (description générale, détails spécifiques, relations entre objets) pour permettre un apprentissage plus structuré.

Intégration avec d'autres datasets

- **Apprentissage multi-source** : Explorer les bénéfices d'un entraînement combinant Flickr 32K avec d'autres datasets comme COCO, Visual Genome ou Conceptual Captions.
- **Validation croisée inter-datasets** : Utiliser d'autres datasets comme ensembles de validation externes pour évaluer la généralisation des modèles entraînés sur Flickr 32K.

— **Transfert de domaine** : Étudier les techniques de transfert d'apprentissage pour adapter les modèles entraînés sur Flickr 32K à des domaines spécifiques (médical, satellite, etc.).

— **Méta-analyse comparative** : Conduire une méta-analyse systématique des différences structurelles entre Flickr 32K et d'autres datasets pour identifier les forces et faiblesses spécifiques.

12 Annexes : Les références

— https://www.analyticsvidhya.com/blog/2021/12/step-by-step-guide-to-build-ima
— https://medium.com/@raman.shinde15/image-captioning-with-flickr8k-dataset-bl
— https://arxiv.org/abs/1706.03762
— https://fairyonice.github.io/Develop_an_image_captioning_deep_learning_model_using_Flickr_8K_data.html
— https://keras.io/examples/vision/image_captioning/